MAL2020 COMPUTING GROUP PROJECT

FINAL REPORT OF

INTERNSHIP MANAGEMENT SYSTEM

Supervisor: Mr Eric Kong Kok Wah

Email: eric.kong@peninsulamalaysia.edu.my

Prepared By:

NAME	COURSE	STUDENT ID
PAVITHIRA A/P PARAMAKURUBARAN	BSSE	BSSE2506005
NEOH JIA YI	BSSE	BSSE2506008
VARSHINI A/P SEGAR	BSSE	BSSE2506011
THULASI A/P G VIJAYARAJ	BSSE	BSSE2509287
DHEVESSHWAAR PUNNIA	BSCS	BSCS2509282

Table of Contents

1.0 Introduction 6

1.1 Overview 6

1.2 Purpose and Objectives 6

1.3 Target Users and Stakeholders 6

1.4 Scope and Limitations 7

1.5 Structure and Organisation of the Report 7

2.0 Background 8

2.1 Problem Domain 8

2.2 Limitations of Existing Solutions 8

2.3 Motivation, Importance, and Relevance 9

2.3.1 Motivation for Development 9

2.3.2 Importance and Relevance 9

2.4 Legal, Social and Professional Issues 10

2.4.1 Legal Issues 10

2.4.1.1 Data Protection and Privacy 10

2.4.1.2 Intellectual Property 10

2.4.1.3 Employment and Anti-Discrimination Law 10

2.4.2 Social Issues 11

2.4.2.1 Digitalising the Internship Process 11

2.4.2.2 Potential Social Impacts 11

2.4.3 Ethical Issues 12

2.4.3.1 Data Privacy and Confidentiality 12

2.4.3.2 Transparency and Fairness in AI Usage 12

2.4.4 Professional Issues 13

2.4.4.1 Adherence to Professional Standards 13

2.4.4.2 Accessibility and Inclusive Design 13

3.0 Requirements 14
3.1 Requirements Gathering Techniques 14
3.2 Stakeholder Involvement in Requirements Definition 15
3.3 Requirement Prioritisation Approach 15
3.4 Functional Requirements 16
3.4.1 Core System Features and Functionalities 16
3.4.2 User Interactions and System Behaviour 16
3.4.3 Input Validation and Output 17
3.5 Non-Functional Requirements 18
3.5.1 Security Requirements 18
3.5.2 Performance and Reliability Requirements 18
3.5.3 Usability and Accessibility Requirements 19
3.5.4 Scalability and Maintainability Considerations 19
3.6 System Modelling 20
4.0 Project Management 21
4.1 Development Methodology 21
4.2 Communication and Collaboration Strategy 22
4.3 Task Allocation and Issue Resolution 22
4.3.1 Task Assignment and Monitoring 22
4.3.2 Issue Escalation and Resolution 22
4.4 Documentation and Knowledge Sharing 22
4.5 Risk Assessment 23
5.0 Development Approach 24
5.1 System Development Strategy 24
5.2 System Architecture and Components 25
5.2.1 User Interface Layer 25
5.2.2 Backend and Business Logic Layer 27

5.2.3 Database and Data Storage Layer 27
5.2.4 Component Integration and Data Flow 27
5.3 Technologies and Tools 28
5.3.1 Programming Languages 28
5.3.2 Frameworks and Libraries 28
5.3.3 Development Environment and Version Control 28
5.3.4 Testing and Deployment Environment 28
5.4 Data Management 29
5.4.1 Database Design and Structure 29
5.4.2 Data Security and Access Control 29
6.0 Project Development Stages 30
6.1 Stage 1 – Preparation and Planning 30
6.2 Stage 2 – Pipeline Design and Feedback Collection 30
6.3 Stage 3 – User Interface (UI) Development 31
6.4 Stage 4 – Business Logic and Backend Integration 31
6.5 Stage 5 – User Testing and Validation 31
6.6 Stage 6 – System Review and Overview 31
6.7 Project Timeline and Gantt Chart 32
7.0 Software Engineering Principles 33
7.1 Application of Software Engineering Principles 33
7.2 Modular and Reusable System Design 34
7.3 System Architecture 35
7.4 Database Schema and ERD 36
7.5 Data Flow and System Behaviour 37
7.6 System Diagrams 38
7.7 Version Control Practices 42
7.8 Code Review and Testing Procedures 42

7.8 Documentation and Maintainability Practices 43
8.0 Quality Assurance 44
8.1 Quality Assurance Strategy 44
8.2 Testing Planning and Execution 44
8.3 Testing Strategies 45
8.3.1 Unit Testing 45
8.3.2 Integration Testing 45
8.3.3 System and Acceptance Testing 46
8.4 Usability and Accessibility Testing 47
8.4.1 User Interface Testing 47
8.4.2 Accessibility Compliance Testing 47
8.4.3 User Feedback and Evaluation 47
10.0 Conclusion 49
10.1 Project Summary and Achievements 49
10.2 Evaluation of Objectives 49
10.3 Final Remarks 49
References 50
Appendices 51
11.1 Student Portal 51
11.2 Employer Portal 59
11.3 Admin Portal 61

1.0 Introduction

1.1 Overview

The Internship Management System (IMS) is a centralised web-based platform developed to automate and streamline the internship management process. By replacing traditional manual methods such as email correspondence and spreadsheet-based tracking, the IMS provides a unified digital environment that connects students, employers, and educational administrators, bridging the gap between academic preparation and professional readiness.

1.2 Purpose and Objectives

The primary purpose of the system is to replace outdated, manual internship tracking with a centralized, AI-powered platform that actively supports all stakeholders throughout the placement process. The system is guided by four specific objectives.

The first objective is process centralisation, which aims to provide students with a unified platform for discovering and tracking internship opportunities, thereby eliminating dependence on disjointed communication channels and manual administrative methods. Second, it seeks to help students with their resume and readiness through an AI-driven resume builder and assessment tools that evaluate fluency, knowledge, and experience. Third, the system is designed to make hiring easier for employers by enabling them to seamlessly post jobs and review applicants within a single platform. Finally, it provides administrators with better monitoring tools to manage students and employers, monitor internship outcomes, and generate detailed analytical reports for data-driven decision-making.

1.3 Target Users and Stakeholders

The system is explicitly designed for three primary target users, each playing a critical role in the internship lifecycle. Students use the platform for readiness assessments, resume generation, job browsing, and application tracking. Employers interact with the system to post vacancies, review applicants, and manage interview scheduling. Administrators oversee the entire platform, managing user accounts and monitoring placement outcomes through real-time analytics.

1.4 Scope and Limitations

The scope of the project encompasses the core digital functionalities required to facilitate the internship lifecycle from preparation to application. This includes the development of the student readiness assessments, AI-enhanced resume generation, employer job posting boards, applicant review interfaces, and administrative oversight modules. As an academic group project, the system is constrained by development timeframes and operates as a self-contained web platform.

To ensure system stability and adherence to these constraints, several advanced features are explicitly out of scope. The platform focuses on standard applicant review and will not utilize AI for automated candidate ranking. Built-in video conferencing for interviews is excluded; instead, the system will support standard interview scheduling. Additionally, the platform will operate independently and will not synchronize with external, third-party job portals such as JobStreet or LinkedIn. Advanced predictive analytics for forecasting student placement success rates will not be implemented in this version. Finally, the AI resume builder requires manual student input and will not feature automated resume parsing or data scraping from pre-existing uploaded documents.

1.5 Structure and Organisation of the Report

This report follows the software development lifecycle. Chapter 2 establishes the background and problem context. Chapter 3 defines the system requirements, outlining what the system must deliver and the constraints governing its development. Chapter 4 covers the project management approach, detailing the Agile methodology and team coordination strategies adopted to meet those requirements. Chapter 5 describes the development approach, including system architecture, technology selection, and data management. Chapter 6 details the sprint-based development stages and project timeline. Chapters 7 and 8 address software engineering principles and quality assurance respectively, covering system design, testing procedures, and validation outcomes. Chapter 9 presents the conclusion, reflecting on project outcomes and future enhancements. Chapters 10 and 11 provide the reference list and appendices.

2.0 Background

2.1 Problem Domain

The IMS is situated within the academic internship and professional placement domain, specifically the transition period during which students move from higher education into the professional workforce. This domain involves continuous coordination between educational institutions, students building professional profiles, and employers seeking qualified interns. Managing this ecosystem requires a reliable communication channel and structured workflows across all three parties.

2.2 Limitations of Existing Solutions

Prior to the conceptualisation of the IMS, the management of academic internships within many institutions relied heavily on fragmented and highly manual administrative processes. The prevailing reliance on disparate tools such as isolated email communications, paper-based documentation, and disconnected spreadsheets created significant administrative burdens and drastically limited the scalability of internship programmes. These approaches created significant administrative burdens, introduced data entry errors, and severely limited the visibility administrators had over student placement progress.

Furthermore, critical gaps existed in how current solutions served end-users. Students frequently faced challenges in adequately preparing for the professional landscape, as they often lacked access to structured, academic-focused tools to assess their readiness or guide them in formulating industry-standard resumes. Conversely, employers were burdened by inefficient recruitment pipelines, expending significant time and resources manually reviewing disorganised and inconsistently formatted applications through disjointed channels. This lack of a unified infrastructure made the hiring process unnecessarily time-consuming and opaque.

2.3 Motivation, Importance, and Relevance

2.3.1 Motivation for Development

The development of the Internship Management System (IMS) was driven by the need to eliminate inefficiencies inherent in manual internship tracking and fragmented data management practices. Traditional approaches often rely on static record-keeping, limiting accessibility, scalability, and actionable insights (Gakii and Kihara, 2026). This project therefore aims to transition towards a centralised, intelligent web-based platform that enhances both operational efficiency and user engagement.

A key focus is improving student readiness through AI-driven tools, including structured assessments and an AI-assisted resume generator. These features support personalised feedback and align with the growing role of artificial intelligence in education and employability (Holmes et al., 2019). The system also aims to streamline recruitment processes for employers and provide administrators with data-driven monitoring of key performance indicators.

2.3.2 Importance and Relevance

The IMS is highly relevant as it addresses a critical gap in the digital transformation of academic internship management. For students, the platform improves employability by offering integrated tools to create professional, dynamically generated resumes and track application progress efficiently. For employers, it provides a streamlined interface for posting opportunities and managing applicants, significantly reducing administrative overhead and improving recruitment efficiency.

From an institutional perspective, the system replaces fragmented and anecdotal tracking methods with real-time analytics and reporting capabilities. Features such as monitoring application rates, assessment completion, and placement outcomes enable data-driven decision-making and improved oversight. Consequently, the IMS contributes to a more transparent, efficient, and coordinated internship ecosystem aligned with modern industry and educational demands.

2.4 Legal, Social and Professional Issues

2.4.1 Legal Issues

2.4.1.1 Data Protection and Privacy

The IMS processes personal data from students, employers, and administrators, including academic records, resumes, and application details. This creates legal obligations under regulations such as the General Data Protection Regulation (GDPR), which mandates principles including data minimisation, purpose limitation, and secure storage (Wolford, 2025).

In accordance with these requirements, the system was designed to collect only the essential data for its intended functionality. Access to student information is governed by role-based permissions, ensuring that only authorised parties are granted access to the data pertinent to their respective roles. For instance, employers reviewing relevant applications. All data is securely stored and protected from unauthorised access or disclosure.

2.4.1.2 Intellectual Property

The IMS integrates the OpenAI API to support AI-assisted resume generation. However, all content is user-provided, with the AI only structuring the information into a professional format. As a result, full intellectual ownership remains with the student. Additionally, all third-party technologies, including React, TypeScript, and Node.js, are used in compliance with their respective licensing terms, and the OpenAI API is implemented according to its official usage policies.

2.4.1.3 Employment and Anti-Discrimination Law

As the IMS incorporates a recruitment pipeline through which employers evaluate and respond to student applications, it was essential to ensure that this process operates within the boundaries of employment and anti-discrimination legislation. Employers are prohibited from making decisions on the basis of protected characteristics such as gender, race, religion, or disability. The employer-facing interface was therefore designed to facilitate structured, transparent, and merit-based hiring interactions, ensuring that all application decisions are grounded in the student's qualifications and suitability for the role.

2.4.2 Social Issues

2.4.2.1 Digitalising the Internship Process

Prior to the introduction of the IMS, the management of the internship process was conducted through largely manual and fragmented methods, resulting in inefficiencies, inconsistencies, and a lack of centralised oversight. The IMS was developed to address these systemic issues by digitalising and centralising the entire internship lifecycle within a single, unified platform.

The system enables college administrators to monitor student progress comprehensively, providing visibility into whether students have submitted applications, the documents and resumes they have uploaded, and the status of their daily reports, all accessible from a single dashboard. This significantly reduces the administrative burden associated with manual tracking and enhances institutional oversight of the placement process.

For students, the platform streamlines the internship experience by consolidating all relevant activities, including internship applications, AI-assisted resume generation, and daily report submissions, into one accessible environment. The AI resume builder guides students through entering their own personal and academic details, after which the OpenAI API generates a professionally formatted resume, thereby supporting students who may have limited familiarity with professional documentation standards.

2.4.2.2 Potential Social Impacts

While the IMS offers substantial social benefits, the development team also considered the potential adverse implications of the system. As the quality and accuracy of the AI-generated resume is directly contingent upon the information provided by the student, incomplete or inaccurate inputs may result in a resume that does not fully represent the student's capabilities. To mitigate this, the system allows students to revisit and update their submitted information, upon which the AI will regenerate the resume based on the revised details, ensuring that the final output remains an accurate and comprehensive reflection of the student.

2.4.3 Ethical Issues

2.4.3.1 Data Privacy and Confidentiality

The IMS handles a substantial volume of sensitive personal and academic data submitted by students in trust. The development team upheld a strong ethical obligation to ensure that this data is managed with the utmost responsibility and confidentiality. Role-based access control was implemented throughout the system to strictly enforce data boundaries. Employers are permitted to access only the student information directly relevant to the application under review, while administrators are restricted to data within the scope of their operational responsibilities. This ensures that no user can access information beyond their designated permissions.

2.4.3.2 Transparency and Fairness in AI Usage

Given the integration of AI within the resume generation process, it was considered ethically essential that the feature operates transparently and equitably. Students are informed that their resume is produced entirely on the basis of the information they provide, ensuring that the process is fully transparent and that no undisclosed logic or external data influences the output. This approach guarantees that the AI serves as an assistive tool rather than an opaque mechanism, maintaining student trust and confidence in the system.

The system also ensures that employer communications with students are conducted in a clear and professional manner. Upon an employer making a decision regarding a student's application, the student is notified promptly. Interview scheduling is managed through a structured process within the platform, providing clarity and transparency for both parties throughout the recruitment lifecycle.

2.4.4 Professional Issues

As computing students engaged in a substantive software development project, the team acknowledged a professional obligation to produce a system of high quality, reliability, and integrity. This section describes the professional standards and practices upheld throughout the development process.

2.4.4.1 Adherence to Professional Standards

Throughout the project, established software engineering best practices were followed to ensure a clean, structured, and maintainable codebase. GitHub was used for version control, enabling systematic tracking of changes and maintaining code stability. Visual Studio Code served as the primary development environment, supporting efficient team collaboration.

All identified defects and security issues were documented and resolved promptly during testing to ensure system reliability. XAMPP was utilised as a local testing environment to validate functionality and manage the database prior to deployment, ensuring that all features met required quality standards.

2.4.4.2 Accessibility and Inclusive Design

The development team recognised a professional responsibility to ensure that the IMS is accessible and usable by all intended users, irrespective of their level of technical proficiency. The student, employer, and administrator interfaces were designed with clarity, consistency, and intuitive navigation as core design principles, ensuring that users are able to interact with the system effectively without requiring specialist technical knowledge.

Attention was given to maintaining a consistent design language and interface structure across all three portals, minimising cognitive load and ensuring that users with varying levels of digital literacy are able to navigate the system with confidence and ease.

3.0 Requirements

This chapter defines the complete set of requirements that guided the design and development of the Internship Management System (IMS). It outlines the techniques used to gather requirements from stakeholders, describes the functional behaviours the system must exhibit, and specifies the non-functional qualities the system must uphold. The requirements documented here formed the foundation for all subsequent design, development, and testing activities throughout the project lifecycle.

3.1 Requirements Gathering Techniques

Requirements for the IMS were gathered using a combination of techniques to ensure a thorough and accurate understanding of user needs. Client meetings were the primary method, in which the development team held structured discussions with the project supervisor to identify the core problems with existing internship management processes and define the expected outcomes of the new system. These sessions were supplemented by direct observation of the current manual workflow, allowing the team to identify specific inefficiencies such as fragmented email communication and the lack of centralised student tracking.

Document analysis was also employed, whereby the team reviewed existing academic internship guidelines and administrative procedures to understand the formal processes the system would need to support. Informal interviews were conducted with prospective users, including students and administrators, to gather first-hand insight into their expectations and pain points. The knowledge collected through these techniques was then consolidated and reviewed collaboratively during sprint planning sessions to produce a finalised and prioritised requirements list.

3.2 Stakeholder Involvement in Requirements Definition

Stakeholder involvement was maintained throughout the requirements definition process to ensure that the system accurately reflected the real needs and expectations of all intended users. Students provided input on preparation features, including the resume builder and application tracking. Employers identified the need for streamlined vacancy posting and applicant review. Administrators specified requirements for the oversight dashboard, including progress monitoring and analytical reporting. Stakeholder feedback was incorporated iteratively after each sprint demonstration.

Stakeholder feedback was incorporated iteratively, with requirements reviewed and refined after each sprint demonstration. This approach ensured that the defined requirements remained aligned with actual user expectations throughout the development lifecycle, reducing the risk of delivering features that did not meet real-world needs.

3.3 Requirement Prioritisation Approach

Requirements were prioritised using the MoSCoW framework to ensure focused and efficient allocation of development effort within project constraints. This approach clearly distinguished critical features from secondary enhancements, supporting structured decision-making.

Must Have requirements define the system's core functionality, while Should Have features enhance usability and operational efficiency. Could Have requirements were identified but deferred due to scope limitations. Lastly, Won't Have requirements for this version were intentionally excluded to maintain development focus and feasibility.

Priority	Requirements
Must Have	Role-based authentication; AI resume builder; internship vacancy posting; administrative analytics dashboard
Should Have	Interview scheduling; applicant status notifications
Could Have	Video interview integration; predictive placement analytics
Won't Have	Third-party job portal synchronisation; automated resume parsing from uploaded documents

Table 3: MoSCoW Priority Classification of System Requirements

3.4 Functional Requirements

Functional requirements define the specific behaviours, features, and operations that the IMS must be capable of performing.

3.4.1 Core System Features and Functionalities

The system must provide role-based authentication for students, employers, and administrators. The student portal must allow users to complete three AI-driven readiness assessments (English fluency, domain knowledge, professional experience) and generate a formatted PDF resume via OpenAI integration using submitted profile data. Students must be able to browse vacancies, submit applications, track application statuses, and view scheduled interviews.

The employer portal must allow companies to create, edit, and manage vacancy listings with attributes including position title, internship type, required skills, and allowance. Employers must be able to view and update application statuses, schedule interviews with shortlisted candidates, and generate employment reports.

The administrator dashboard must support full user account management, KPI monitoring (total applied interns, assessment completion rates), analytical report generation, automated reminder posting, and new administrator account creation.

3.4.2 User Interactions and System Behaviour

Upon login, the system must validate credentials and route each user to their respective portal. After a student completes all assessments and submits their profile, the system must call the OpenAI API and return a downloadable resume; a loading indicator must be displayed during this process. Application submissions must be confirmed to the student, must prevent duplicates to the same vacancy, and must be immediately visible to the relevant employer.

When an employer updates an application status to Shortlisted, Accepted, or Rejected, the student must be notified promptly. Interview scheduling must generate confirmation for both parties. The administrator dashboard KPIs must refresh dynamically without requiring a manual page reload.

3.4.3 Input Validation and Output

The system accepts input data including student registration details, readiness assessment responses, personal profiles, employer credentials, vacancy listings, and application submissions. All inputs are validated on both client and server sides to ensure data integrity. Required fields must be completed before submission, formats (e.g., email addresses) must follow standard conventions, and text inputs are constrained by appropriate character limits.

The system processes student assessment results and profile data via the OpenAI API to generate a structured PDF résumé as the primary output. Additional outputs include real-time application status updates for students and employers, published vacancy listings, and administrative analytics reports. All outputs are presented in a clear, structured format tailored to the user’s role and technical proficiency.

3.5 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints that the system must satisfy. These requirements govern how the system performs its functions rather than what specific functions it performs, and are equally critical to the overall success of the platform.

3.5.1 Security Requirements

The system must enforce strict security controls to protect sensitive user data and prevent unauthorised access. All user passwords must be stored in a hashed format using an industry-standard algorithm and must never be stored as plain text. Authentication must be managed through JWT, ensuring that only verified, active sessions can access protected API endpoints. Role-based access control (RBAC) must be applied consistently, ensuring that students, employers, and administrators can only access the data and functionality appropriate to their respective roles. All client-server communication must be conducted over HTTPS to prevent data interception. The system must also implement parameterised SQL queries throughout the data layer to protect against SQL injection vulnerabilities.

3.5.2 Performance and Reliability Requirements

The system must respond to standard user interactions, such as page navigation, form submissions, and data retrieval, within an acceptable timeframe to maintain a fluid and professional user experience. Standard API calls to the backend must return responses promptly, while AI-powered resume generation, which relies on an external API call to OpenAI, is acknowledged to involve a longer processing duration; the system must display an appropriate loading indicator during this process to manage user expectations.

The platform must operate reliably without unexpected failures during normal use, and all data submitted by users must be persisted accurately and consistently in the relational database. Error handling mechanisms must be implemented throughout the system to manage unexpected failures gracefully, displaying informative messages to the user rather than exposing raw technical errors.

3.5.3 Usability and Accessibility Requirements

The system must be intuitive and easy to use for individuals with varying levels of technical proficiency. All three user portals must feature a consistent design language and navigation structure, enabling users to locate required features without extensive training. The interface must be fully responsive, rendering correctly and remaining fully functional across desktop and mobile web browsers.

All interactive elements, including form fields, buttons, and navigation links, must include descriptive labels to support screen readers and assistive technologies. Color contrast ratios across the interface must meet general web accessibility standards, ensuring readability for users with visual impairments. Inline validation feedback must be provided on all forms to guide users in correcting input errors before submission.

3.5.4 Scalability and Maintainability Considerations

The system shall be designed to support increasing volumes of users, vacancies, and application data without significant degradation in performance. This is achieved through a three-tier architecture combined with RESTful API design principles, enabling independent scaling of system components in response to demand.

To ensure long-term maintainability, the system shall adopt a modular code structure that allows new features and enhancements to be implemented with minimal impact on existing functionality. Consistent coding standards, clear naming conventions, and comprehensive documentation shall be enforced to support efficient system maintenance and developer onboarding.

Additionally, the database design shall accommodate data growth and future extensibility, ensuring that new functionalities, such as predictive analytics or video interview integration, can be incorporated without requiring significant restructuring of the existing schema.

3.6 System Modelling

System modelling was utilised to provide clear visual representations of the IMS structure and behaviour prior to implementation. These models established a shared understanding among stakeholders and developers, serving as a blueprint that guided design decisions and reduced ambiguity.

A set of UML diagrams was developed to capture both structural and behavioural aspects of the system. The Use Case Diagram defined stakeholder interactions, the Entity Relationship Diagram (ERD) modelled core data entities and relationships to inform database design, and the Class Diagram represented the system's static structure. Sequence and Activity Diagrams illustrated key workflows, including AI résumé generation and the internship application process. Full diagrams are provided in Chapter 7.

4.0 Project Management

Effective project management was fundamental to ensuring that the Internship Management System was delivered in a structured, collaborative, and timely manner. This chapter outlines the methodology and communication practices adopted by the development team during the current development phase, covering the Agile framework that guided the team's workflow and the collaborative tools and practices used to coordinate development activities.

4.1 Development Methodology

The development of the IMS was guided by the Agile software development methodology. It is an iterative and incremental approach that emphasises adaptability and delivery of functional software through structured development cycles known as sprints (Sommerville, 2011). The team adopted Agile as it was well-suited to the nature of the project, which involved three distinct user roles whose requirements evolved progressively as development advanced.

The project was structured across six sprints, each targeting a specific functional area:

Sprint	**Focus Area**
Sprint 1	Secure login and role-based access control
Sprint 2	AI-assisted resume builder
Sprint 3	Job posting and application management
Sprint 4	Interview scheduling
Sprint 5	Analytics and administrative reporting
Sprint 6	System integration and comprehensive testing

Table 1: Sprint Plan

This sprint-based structure ensured that development remained focused and that progress was measurable at the conclusion of each cycle. At the conclusion of each sprint, the team conducted a sprint review to evaluate completed work, reflect on challenges encountered, and determine priorities for the subsequent sprint. This practice of regular retrospection enabled the team to maintain a high standard of quality and continuously refine both the system and the team's working processes throughout the development lifecycle.

4.2 Communication and Collaboration Strategy

WhatsApp served as the primary communication platform for daily coordination, enabling rapid exchange of updates and resources. Task allocation was managed collaboratively during in-class sessions, where members assessed progress and distributed responsibilities based on individual capacity and expertise. Regular meetings with the project supervisor and client allowed the team to demonstrate completed features, receive structured feedback, and ensure ongoing alignment with client requirements.

4.3 Task Allocation and Issue Resolution

4.3.1 Task Assignment and Monitoring

Task allocation and progress tracking were managed through real-time communication via a dedicated WhatsApp group. Tasks, including system modules and documentation, were assigned based on individual strengths and availability. Progress updates were shared continuously within the group, enabling transparent monitoring of workload distribution and ensuring alignment with sprint deadlines. This continuous stream of communication ensured that all team members remained highly aware of the current development status, individual workload capacities, and approaching sprint deadlines.

4.3.2 Issue Escalation and Resolution

During the development lifecycle, technical issues, bugs, and integration challenges were resolved through a collaborative escalation process. Problems were reported immediately in the group chat and addressed collectively through peer discussion and debugging support. This approach enabled rapid issue resolution, reduced development delays, and promoted effective team collaboration.

4.4 Documentation and Knowledge Sharing

To ensure seamless collaboration on project documentation, the team utilised an online Word document as the primary repository for written materials. This cloud-based approach allowed multiple team members to access, co-author, and update the final report and system specifications simultaneously in real-time.

For technical assets, all high-fidelity user interface wireframes were created and stored collaboratively using Figma, while the system's codebase was strictly managed and shared via GitHub. By leveraging these online platforms, the team ensured that everyone had immediate, remote access to the most up-to-date project materials, effectively preventing version control conflicts and streamlining the sharing process.

4.5 Risk Assessment

This section identifies potential risks associated with the development of the IMS and the mitigation strategies implemented to reduce their impact on project success. Risks are categorised based on their likelihood and severity to ensure appropriate preventive and corrective measures are applied.

Risk	**Impact**	**Likelihood**	**Mitigation Strategy**
Data Breaches	High	Medium	Role-based logins and JWT authentication implemented for all user types
AI Scoring Inaccuracies	Medium	Low	Continuous testing and validation of the three artificial intelligence driven readiness tests and the five-step automated generation pipeline.
Client Alignment	High	Low	Frequent client meetings with structured progress demonstrations
Scope Creep	Medium	Low	Adherence to strict sprint planning and deferring advanced features like video interviews and predictive analytics to future development phases.
System Integration Failures	High	Low	Continuous integration testing across all three portals throughout development

5.0 Development Approach

This chapter details the technical foundation and architectural design of the Internship Management System. It outlines the strategies, components, and specific technologies chosen to build a scalable and efficient platform for students, employers, and administrators.

5.1 System Development Strategy

The development of the system was driven by a component-based, three-tier architecture that distinctly separates the presentation layer, the application logic layer, and the database layer. Development was organised through the Agile methodology, allowing features to be built in isolated sprints and validated iteratively with the client.

The technical decisions for the system were justified by the need for performance, scalability, and ease of maintainability. React coupled with TypeScript was selected for the frontend presentation layer. This combination was chosen because React supports the creation of highly reusable user interface components, which ensures visual consistency across the three separate user portals, while TypeScript adds strict type checking to minimize runtime errors.

For the backend, Node.js was selected to handle the server-side business logic. Node.js was specifically chosen for its asynchronous, event driven architecture, making it highly efficient at processing the artificial intelligence readiness tests and managing the OpenAI application programming interface integration required for the automated resume builder. This strict separation of technologies ensures the system remains modular, allowing future developers to update the frontend interfaces or backend artificial intelligence logic independently without disrupting the entire platform.

5.2 System Architecture and Components

This section breaks down the architecture of the Internship Management System into its core functional areas, detailing how users interact with the platform, how data is processed, and how information is securely stored.

5.2.1 User Interface Layer

The user interface layer dictates how individuals interact with the system, prioritising screen layouts, navigation flow, and overall usability across all three user portals.

The **student interface** is designed to centralize the internship preparation and application process (Figure 5.2.1.1). The dashboard features intuitive navigation to the artificial intelligence powered resume builder, allowing students to complete three distinct readiness assessments. Additionally, the interface provides a clear view of total jobs applied, upcoming interview schedules, and a progress tracker for assessment completion.

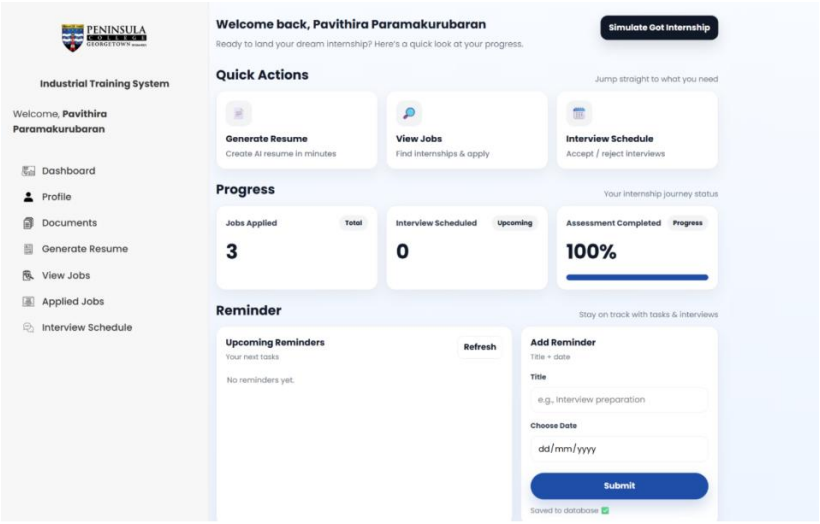


Figure 5.2.1.1 shows the Student Dashboard interface.

The **employer portal** prioritizes efficient recruitment management. The design illustrates a clean interface for companies to seamlessly post detailed internship vacancies and view their company profile (Figure 5.2.1.2). A critical component of this design is the applicant review section, which enables employers to track the number of job applications, scheduled interviews, and completed interviews at a glance.

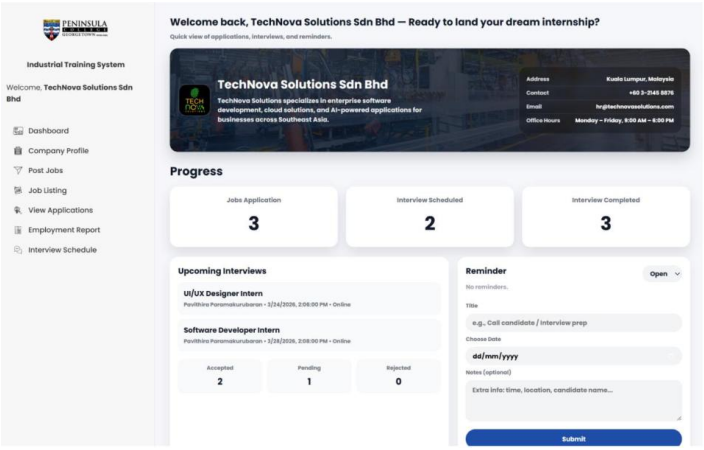


Figure 5.2.1.2 shows the Employer Portal interface.

The **administrator interface** is built for high level oversight and system management (Figure 5.2.1.3). The layout highlights a comprehensive analytics dashboard displaying real time key statistics, such as total registered students, total employers, posted jobs, and total applications. Administrators can easily navigate through the sidebar to manage student and employer accounts, review analytical reports, and add new admins.

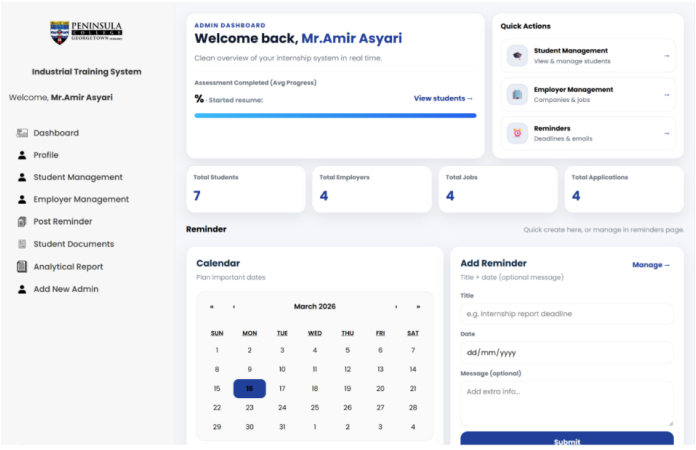


Figure 5.2.1.3 shows the Administrator Dashboard interface.

5.2.2 Backend and Business Logic Layer

The application layer relies on Node.js to handle the server-side logic and process user requests. This layer manages the core business workflows and calculations that support the entire system functionality. It evaluates the three artificial intelligence driven readiness tests and communicates directly with the OpenAI application programming interface to facilitate the automated resume generation pipeline.

Furthermore, the backend enforces system rules by processing internship applications, scheduling interview time slots, and generating real time analytical reports for the administrator dashboard.

5.2.3 Database and Data Storage Layer

The system uses a relational database managed through MySQL via XAMPP. The schema is designed around a central User entity that stores authentication credentials and profile data, with dedicated tables for internship vacancies, applications, assessments, and resumes. Full schema details are provided in Section 7.4.

5.2.4 Component Integration and Data Flow

The integration between these components follows a structured data flow to ensure seamless communication across the platform. When a user interacts with the React frontend interface, the presentation layer sends an asynchronous request to the Node.js backend. The application logic processes this request, validates the input data, and queries the MySQL relational database to store or retrieve the necessary information. Once the database operation is complete, the backend returns a confirmation response to the frontend, dynamically updating the user interface.

5.3 Technologies and Tools

5.3.1 Programming Languages

The system is primarily developed using TypeScript and JavaScript. TypeScript is used in the frontend to enforce static type checking, reducing runtime errors and improving code reliability.

5.3.2 Frameworks and Libraries

React is used as the main frontend framework to develop a responsive, component-based user interface with consistent design across portals. The backend is built using Node.js with Express.js, which handles server-side logic, API routing, and integration of AI services.

5.3.3 Development Environment and Version Control

Visual Studio Code is used as the primary development environment for coding and debugging. GitHub is employed for version control, enabling structured collaboration, branch management, and change tracking (GitHub, 2024).

5.3.4 Testing and Deployment Environment

For testing and database management prior to formal deployment, XAMPP and phpMyAdmin are utilized as the local testing environment to ensure all system functionalities operate correctly and safely before release.

Category	Technology	Purpose
Frontend language	TypeScript / JavaScript	Type-safe UI development
Frontend framework	React (Vite)	Component-based, reactive UI
Backend runtime	Node.js with Express.js	API routing and business logic
AI integration	OpenAI API	Resume generation
Database	MySQL (XAMPP / phpMyAdmin)	Relational data storage
Version control	Git / GitHub	Code management and collaboration
IDE	Visual Studio Code	Development environment
Testing environment	XAMPP / phpMyAdmin	Local database testing and validation

Table 2: Technologies and Tools

5.4 Data Management

5.4.1 Database Design and Structure

The system relies on a relational database managed through XAMPP and MySQL. Data is organized into structured tables to support the complex relationships between students, employers, and administrators. Specific tables are dedicated to storing the artificial intelligence assessment scores for student fluency, knowledge, and experience, which feed directly into the automated resume generation module to ensure outputs are backed by persistent data.

5.4.2 Data Security and Access Control

The platform strictly adheres to data protection requirements by incorporating principles of data minimization and secure data storage. To protect sensitive information, secure role-based logins are implemented for all three user types. Strict system access controls, JSON Web Token (JWT) authentication, and password hashing techniques ensure that only authorized users can view or modify specific records, proactively mitigating the risk of potential data breaches.

6.0 Project Development Stages

The development of the Internship Management System (IMS) followed a structured, Agile-based approach divided into six stages. This ensured systematic planning, iterative delivery, continuous feedback, and quality assurance. Each stage contributed incremental deliverables toward the final system.

6.1 Stage 1 – Preparation and Planning

This stage established the project foundation by defining the scope, requirements, and development roadmap. An analysis of existing internship management practices was conducted to identify key problems and user needs.

Activities included background research on internship systems and AI-assisted recruitment tools, stakeholder identification, and definition of user roles (students, employers, administrators). Functional and non-functional requirements were gathered, and project objectives, scope, and deliverables were defined alongside the selection of appropriate technologies. A structured plan incorporating milestones, sprint allocation, and task dependencies was also developed, with deliverables prioritised using the MoSCoW framework (Section 5.3).

The outputs of this stage, including Project Proposal, Requirement Specification, and preliminary system architecture, provided the basis for all subsequent development.

6.2 Stage 2 – Pipeline Design and Feedback Collection

This stage focused on designing system workflows and data pipelines across the frontend, backend, AI modules, and database. User journeys for all roles were defined, and the internship application lifecycle was mapped from submission to placement.

Early stakeholder feedback was collected to validate and refine workflows before development. Progress was monitored using project tracking tools, and iterative reviews ensured alignment with requirements. This stage ensured that system processes were logically structured and aligned with real-world scenarios.

6.3 Stage 3 – User Interface (UI) Development

Stage 3 involved developing the presentation layer using a user-centred design approach. Wireframes were created in Figma and implemented as responsive React components. Client-side validation and error handling were incorporated, and iterative sprint reviews enabled continuous interface refinement. The outcome was a functional, accessible UI ready for backend integration.

6.4 Stage 4 – Business Logic and Backend Integration

This stage implemented the system's core functionality by integrating the frontend with backend services and the database, while also introducing AI capabilities.

Core backend functionality was implemented, including JWT-based authentication, RBAC, internship posting and application tracking, and user account management. The AI resume builder was integrated with the OpenAI API, and secure database connectivity was established. This stage produced a fully integrated system operating within the three-tier architecture.

6.5 Stage 5 – User Testing and Validation

Unit, integration, and system testing were conducted to verify correctness and reliability. User Acceptance Testing (UAT) involved students and employers completing structured task scenarios. Defects were identified, prioritised, and resolved. Detailed test cases and results are documented in Chapter 8.

This stage confirmed that the system met both technical and user requirements.

6.6 Stage 6 – System Review and Overview

The final stage evaluated the system against project objectives and ensured completion of all deliverables. System performance, security, and AI considerations were reviewed, and all documentation was finalised.

A full system walkthrough confirmed that core features met the defined requirements, and retrospective analysis identified areas for future improvement (see Section 9).

6.7 Project Timeline and Gantt Chart

The six development stages were scheduled and tracked using a Gantt chart (Figure 6.7.1). This project management tool provides a comprehensive visual timeline of the entire development lifecycle. It clearly illustrates the start and end dates for each specific development stage, outlines task dependencies, and serves as a continuous tracking mechanism allowing the team to compare planned milestones against actual progress.

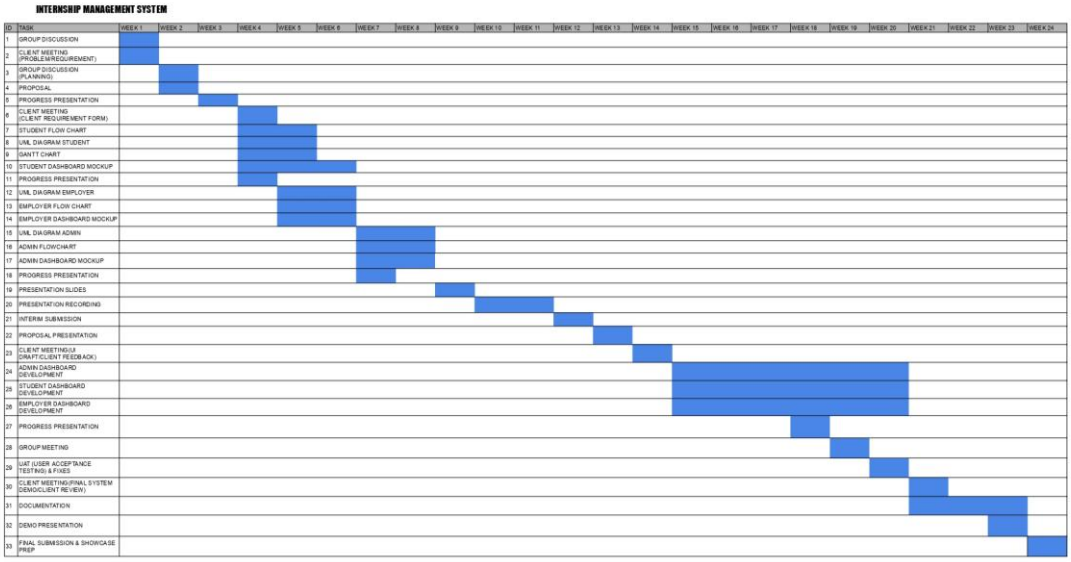


Figure 6.7.1 shows the Project Timeline and Gantt Chart.

7.0 Software Engineering Principles

This chapter outlines the software engineering principles, architectural design, and development practices applied in the IMS.

7.1 Application of Software Engineering Principles

The IMS was developed based on four core principles embedded across all system layers.

Maintainability was achieved through modular design, separation of concerns, and consistent naming conventions. Components were structured around single responsibilities, with organised API routes and standardised coding practices, enabling efficient modification and onboarding.

Scalability was supported by a three-tier architecture and stateless RESTful APIs, allowing independent scaling of frontend, backend, and database layers. Technologies such as React (Vite) and MySQL support performance optimisation and growth in users and data.

Reliability was ensured through server-side validation, structured error handling, and comprehensive testing. Integration testing verified system interactions, while UAT confirmed correct behaviour under real-world conditions.

Security was enforced through RBAC, password hashing, JWT-based authentication, HTTPS communication, and parameterised SQL queries, ensuring protection against unauthorised access and common vulnerabilities.

7.2 Modular and Reusable System Design

The IMS is structured around five independent, single-responsibility modules, each encapsulating a distinct business function (Table 4). This modular approach reduces inter-component coupling, simplifies testing, and supports reuse across user roles.

Module	Responsibility	User Roles
User Authentication	Manages registration, login, session handling, and role-based access control via a unified pipeline	Student, Employer, Admin
Job Posting	Enables employers to create, edit, and manage internship listings; provides students and admins with read access	Student, Employer, Admin
Application Tracking	Handles end-to-end application submission, status management, and employer review workflow	Student, Employer, Admin
AI Resume Builder	Generates personalised PDF resumes from student assessment scores (language, knowledge, experience) and personal profile data via OpenAI API integration	Student
Admin Analytics	Aggregates platform data into real-time KPIs: total users, jobs, applications, resume participation, assessment progress, and platform ratios	Admin

Table 4: IMS Module Overview

The User Authentication module exemplifies the reuse principle, a single login pipeline serves all three user roles, with role-based routing applied post-authentication to direct each user to their respective portal. This eliminates redundant logic and ensures security controls are applied uniformly across the platform.

7.3 System Architecture

The IMS follows a Three-Tier Client-Server Architecture, dividing the system into three logically separated layers (Table 5). This pattern enforces a strict separation of concerns, prevents direct client-to-database communication, and allows each tier to evolve independently.

Layer	Technology	Role	Example
Presentation	React.js (Vite)	User interface rendered in web browser	Login page, dashboard
Application	Node.js with Express.js	Business logic, API routing, authentication and AI integration	/login, /jobs, /applications
Data	MySQL (XAMPP / phpMyAdmin)	Relational data storage and retrieval	Applications, jobs, reports

Table 5: Three-Tier Client-Server Architecture

Communication between layers is handled exclusively through RESTful APIs. The frontend never communicates directly with the database; all requests pass through the backend, which applies validation, business logic, and security controls before executing database operations.

7.4 Database Schema and ERD

The relational schema centres on a User table that stores authentication credentials and role designation, from which three role-specific tables branch which are Student, Employer, and Admin. The core transactional tables are as follows:

Table	Key Fields	Relationships
User	user_id (PK), name, email, password_hash, role	Parent to Student, Employer, Admin
Student	student_id (PK), user_id (FK), course, matric_no	One-to-many with Application, Assessment, Resume
Employer	employer_id (PK), user_id (FK), company_name, industry	One-to-many with Job
Job	job_id (PK), employer_id (FK), title, type, skills, allowance, status	One-to-many with Application
Application	app_id (PK), student_id (FK), job_id (FK), status, submitted_at	Links Student to Job
Assessment	assessment_id (PK), student_id (FK), fluency_score, knowledge_score, experience_score	One-to-one with Student
Resume	resume_id (PK), student_id (FK), generated_content, pdf_path, created_at	One-to-one with Student
Interview	interview_id (PK), app_id (FK), scheduled_at, location, status	One-to-one with Application

The full ERD is presented in the Appendix.

7.5 Data Flow and System Behaviour

System interactions follow a controlled, unidirectional data flow. User actions in the React frontend trigger API requests to the Express backend, which validates inputs, enforces access controls, and executes parameterised SQL queries. The database returns results to the backend, which formats them as structured JSON and sends a response to the frontend, which updates the UI dynamically.

For AI resume generation specifically, the flow is: student submits profile → backend retrieves assessment scores from the Assessment table → constructs a structured prompt → sends a request to the OpenAI API → receives generated resume content → stores it in the Resume table → returns a downloadable PDF to the frontend.

7.6 System Diagrams

The **class diagram** (Figure 7.6a) consists of a base User class with attributes such as ID, name, email, and password, and common methods like login and profile management. It is extended by Student, Employer, and Admin with role-specific attributes and methods. Additional classes include Job, which stores job details, Application, which links Students to Jobs, Assessment for evaluations, and Resume for student profiles. Relationships show that Employers create Jobs, Students submit multiple Applications, and each Student maintains one Resume.

Figure 7.6a: Class diagram

Sequence Diagram (Figure 7.6b) illustrates the step-by-step process of AI Resume Generation, starting when a student initiates the request. The system collects user data, processes it through the AI module, generates the resume content, and converts it into a PDF before delivering it back to the student.

Figure 7.6b: Sequence diagram

Activity Diagram (Figure 7.6c) represents the Internship Application workflow, showing actions such as job selection, application submission, and validation. It includes decision points for checking missing information and preventing duplicate applications before confirming submission. For example, the student selects a vacancy, after which the system validates for duplicate applications. If a duplicate exists, submission is blocked and the student is notified; otherwise, the application is confirmed, stored, and made accessible to the employer.

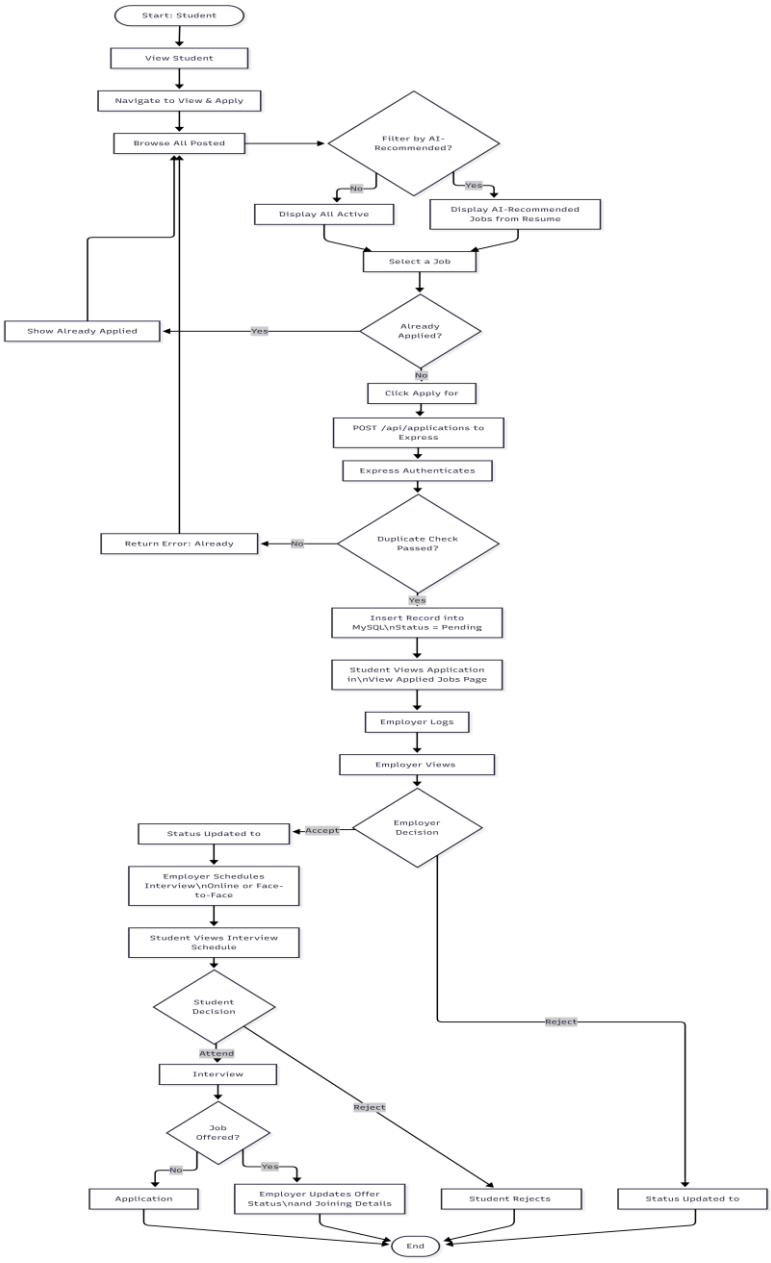


Figure 7.6c: Activity diagram

State Diagram (Figure 7.6d) describes the application lifecycle, progressing from Pending to Under Review, followed by Shortlisted, and concluding with either Accepted or Rejected. A Withdrawn state allows the student to exit the process prior to a final decision.

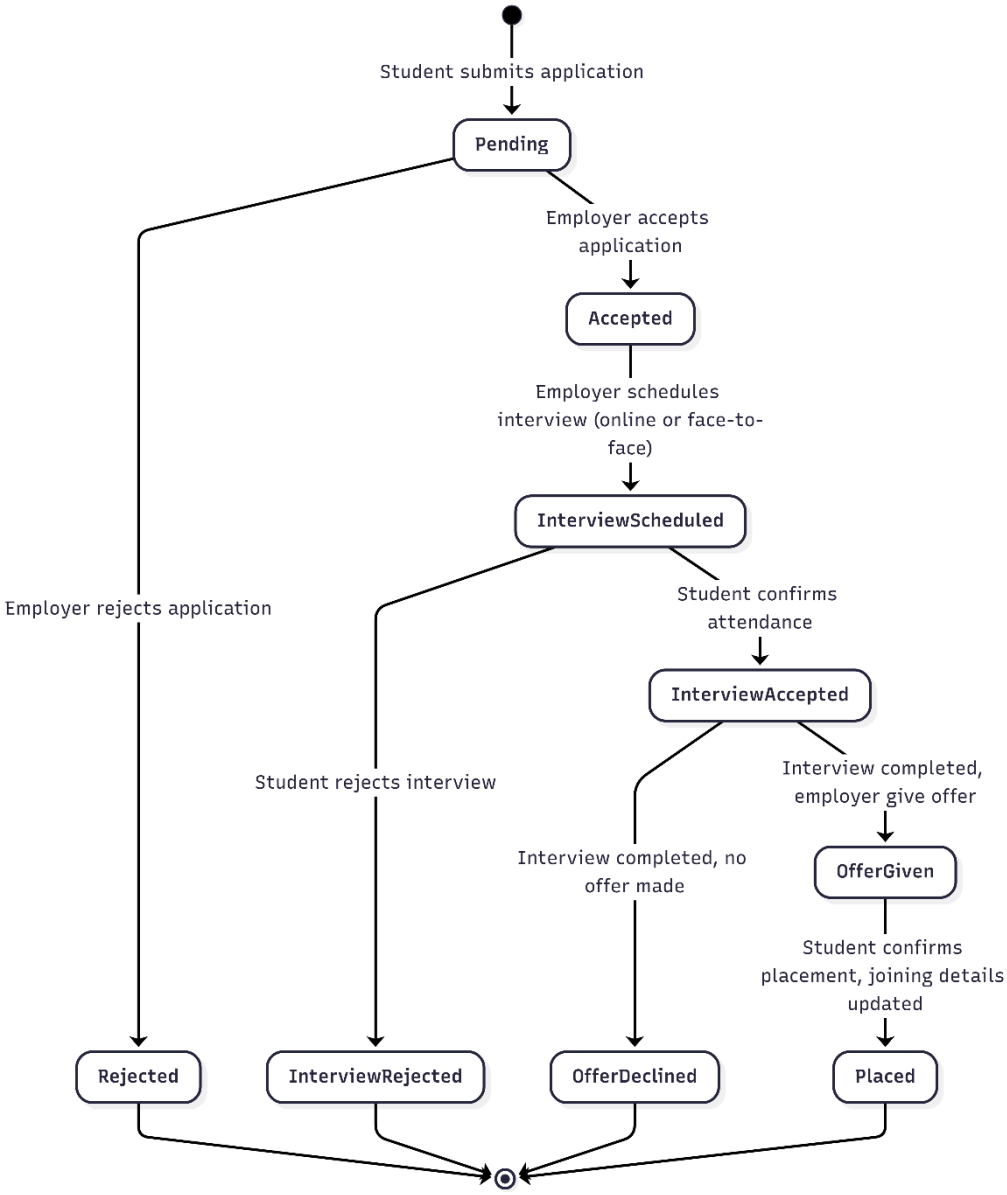


Figure 7.6d: State diagram

7.7 Version Control Practices

Version control was managed using Git and GitHub to ensure reliability and collaboration throughout development. The project followed a monorepo structure with separate directories for the React frontend (/client) and Express backend (/server).

Commits were made at consistent, meaningful intervals, each representing a stable unit of work. This approach provided clear traceability, rollback capability, and an auditable history of changes. GitHub further supported collaboration through off-site backup, issue tracking, and pull request workflows, ensuring code integrity and long-term maintainability.

7.8 Code Review and Testing Procedures

Peer code reviews were conducted consistently throughout development to uphold code quality and maintain structural consistency. These reviews emphasized early error detection, adherence to naming and design conventions, and improvements in readability. They also fostered a shared understanding of cross- cutting components such as the authentication middleware and AI integration module, ensuring that critical system elements were implemented in a uniform and reliable manner.

Testing was applied progressively across three levels to verify correctness and usability. Unit testing focused on individual functions in isolation, including login validation, form submission handlers, and assessment score calculations, with boundary values and both valid and invalid inputs used to confirm expected behavior. Integration testing validated end- to- end flows across the React frontend, Express backend, and MySQL database, with submitted data inspected directly in phpMyAdmin to ensure accurate processing and persistence. Finally, UAT involved real students and employers completing structured task scenarios, such as registration, assessment completion, resume generation, job posting, and application submission, to confirm real- world usability and functional correctness.

Testing was conducted using Google Chrome for UI interaction and manual validation, alongside XAMPP phpMyAdmin for database verification. This combined approach provided comprehensive assurance of system correctness, reliability, and readiness prior to deployment.

7.8 Documentation and Maintainability Practices

Comprehensive documentation was maintained throughout development, including inline code comments, a README file covering environment setup and configuration, and system-level documentation covering architecture, API endpoints, module responsibilities, and database schema. This supports efficient developer onboarding and safe future extension of the system.

8.0 Quality Assurance

This chapter outlines the overarching quality strategy, the structured execution of testing phases, and the specific evaluations conducted to guarantee that the final platform met all functional and non-functional requirements.

8.1 Quality Assurance Strategy

The quality assurance (QA) strategy for the IMS was established to ensure that the final system met rigorous academic and industry standards. The approach focused heavily on defect prevention, continuous process improvement, and strict compliance with the project's defined requirements (Marway, 2023).

Aligned with Agile methodology principles, QA was implemented as an ongoing process rather than a final phase. Quality checklists and explicit acceptance criteria were established for each development sprint, enabling systematic validation of deliverables. Early integration of testing practices ensured that critical components, such as role-based authentication, the AI-powered résumé builder, and the analytics dashboard, were thoroughly verified before subsequent modules were developed and integrated.

This proactive and iterative QA approach reduced defect propagation, improved system reliability, and ensured consistent alignment with project objectives throughout the development lifecycle.

8.2 Testing Planning and Execution

To ensure comprehensive defect detection, testing planning and execution were formally structured. The test plan defined exactly what required testing, the methods to be used, and the timeline for execution, which culminated primarily during Sprint 6. Test cases were meticulously prepared to cover all user workflows for students, employers, and administrators.

The execution phase involved a deliberate progression from testing small, isolated components to evaluating the entire interconnected system. By utilizing XAMPP as the local testing environment, the team safely identified and resolved database relationship errors and logic defects prior to any formal deployment, ensuring a stable release.

8.3 Testing Strategies

8.3.1 Unit Testing

Unit testing was conducted to verify that individual components and isolated functions of the IMS operated correctly at the codebase level. Representative test cases are shown below:

Test ID	Component	Input	Expected Output	Result
UT-01	Login validation	Valid student email + correct password	Redirect to student dashboard	Pass
UT-02	Login validation	Valid email + incorrect password	Error message displayed; no redirect	Pass
UT-03	Login validation	Empty email field	Client-side validation error before submission	Pass
UT-04	Assessment score calculation	Fluency = 80, Knowledge = 70, Experience = 60	Correct weighted scores stored in Assessment table	Pass
UT-05	Resume generation trigger	Complete student profile + all three assessment scores present	OpenAI API called; resume content returned and stored	Pass

Table 6

8.3.2 Integration Testing

Following the successful validation of individual units, integration testing was performed to evaluate the interactions between different integrated modules. This testing phase was crucial to ensure that data flowed correctly across the system's three-tier architecture.

Key tests included:

Test ID	Integration Point	Scenario	Expected Behaviour	Result
IT-01	Frontend → Backend → DB	Student submits application form	Application record created in DB; confirmation shown in UI	Pass
IT-02	Backend → OpenAI API → DB	Resume generation request submitted	API response received; content stored in Resume table; PDF returned	Pass
IT-03	Employer portal → DB	Employer updates	Status updated in Application table; student notified in UI	Pass

		application status to Accepted		
IT-04	Admin dashboard → DB	Admin views KPI dashboard	Correct totals for students, employers, jobs, and applications pulled from DB	Pass
IT-05	Auth middleware	Request to protected route without valid JWT	401 Unauthorised response returned	Pass

Table 7

8.3.3 System and Acceptance Testing

System testing was executed to verify the complete, fully integrated web platform against the original Requirement Specification document. This involved end-to-end testing of the entire internship lifecycle. Following system verification, UAT was conducted to confirm that the platform met the practical expectations of its intended users. During UAT, sample users simulated real-world scenarios, such as a student registering an account, completing the readiness tests, generating a resume, and successfully applying for an internship vacancy, followed by an employer receiving and reviewing that specific application. This confirmed the system's readiness for actual deployment.

Scenario	User	Steps	Expected Outcome	Result
UAT-01: Full student journey	Student	Register → complete assessments → generate resume → browse jobs → apply	All steps completed; resume downloaded; application visible to employer	Pass
UAT-02: Employer review workflow	Employer	Log in → view applications → shortlist candidate → schedule interview	Application status updated; interview confirmation visible to student	Pass
UAT-03: Admin monitoring	Admin	Log in → view KPI dashboard → check student management page	Correct user counts and statuses displayed	Pass
UAT-04: Duplicate application prevention	Student	Attempt to apply to the same vacancy twice	Second submission blocked; appropriate message displayed	Pass
UAT-05: Invalid login	All roles	Submit login with wrong password	Error message displayed; session not created	Pass

Table 8

8.4 Usability and Accessibility Testing

8.4.1 User Interface Testing

Rigorous User Interface (UI) testing was conducted across the student, employer, and administrator portals to ensure a seamless and intuitive user experience. The evaluation focused on the visual design, layout consistency, responsiveness, and usability of the React-based components.

Key aspects assessed included the placement and accessibility of navigation elements, the logical flow of application forms, and the readability of analytical reports on the administrative dashboard. Testing was performed on multiple devices and screen sizes using Google Chrome, confirming that all interfaces rendered correctly and remained fully functional across both desktop and mobile viewports.

8.4.2 Accessibility Compliance Testing

Accessibility compliance testing was integrated to verify that the system could be used effectively by individuals with varying levels of digital literacy and visual needs.

Adhering to general web content accessibility principles, the team evaluated the platform to ensure proper color contrast ratios between text and background elements. Furthermore, the interfaces were tested to guarantee that keyboard navigation functioned correctly, ensuring that all interactive elements, such as the interview scheduling calendar and the resume builder form, were fully accessible without strict reliance on a mouse.

8.4.3 User Feedback and Evaluation

The final component of the usability strategy involved gathering direct user feedback and evaluation. Following system demonstrations, feedback was collected from individuals representing the target user groups (students and employers) to identify any remaining usability friction or workflow confusion. The insights gathered from these feedback sessions were highly valuable, allowing the development team to make final refinements to the user interface, clarify navigation paths, and ensure the final product provided an optimized and highly supportive user experience.

9.0 Future Enhancements

While the current system provides a comprehensive and functional solution for internship management, several advanced features were identified during the requirements gathering phase that were intentionally deferred to future development cycles. This decision was made to ensure system stability and to allow the core functionalities to be thoroughly tested and refined before introducing additional complexity. As the platform matures and user needs evolve, these enhancements will serve as the foundation for the next phase of development.

The first planned enhancement involves integrating **native video interview capabilities** through platforms such as Zoom or Microsoft Teams, enabling employers and students to conduct interviews directly within the system without relying on external applications. Alongside this, **advanced Natural Language Processing (NLP) models** will be incorporated to deliver more accurate and in-depth evaluations of student resumes, moving beyond basic keyword matching to a deeper understanding of candidate profiles. To support data-driven decision-making, **predictive analytics** will be introduced to forecast student placement success rates by analyzing historical patterns using machine learning techniques. Finally, **automated resume parsing** will enable the system to intelligently extract structured information such as the applicant's name, skills, educational background, and work history directly from uploaded resume documents, streamlining the onboarding process and improving overall data accuracy across the platform.

10.0 Conclusion

This chapter summarizes the overall achievements of the Internship Management System project, evaluates the extent to which the initial objectives were met, and outlines potential future enhancements for the platform.

10.1 Project Summary and Achievements

The primary goal of this project was to address the inefficiencies of traditional manual internship management methods, which relied heavily on fragmented emails and disjointed spreadsheets. By developing a centralized web platform, the team successfully delivered an intelligent digital solution that bridges the gap between academic preparation and professional readiness. The system successfully implements a secure, three tier architecture ensuring a scalable, reliable, and maintainable environment for all users.

10.2 Evaluation of Objectives

The completed system successfully fulfils its four primary objectives. First, it provides a centralized platform for students to track opportunities and manage applications seamlessly. Second, the integration of artificial intelligence actively assists students by evaluating their English fluency, domain knowledge, and experience through readiness tests, culminating in the automated generation of a professional resume via the OpenAI application programming interface. Third, the employer portal streamlines the recruitment process by allowing companies to post vacancies, review applicants, and schedule interviews within a single interface. Finally, the system equips academic administrators with robust monitoring tools, providing real time analytical reports and key performance indicators to oversee the entire placement ecosystem effectively.

10.3 Final Remarks

Ultimately, the Internship Management System proves that digitalizing the academic placement process significantly reduces administrative burdens while actively empowering students in their professional transition. The project adheres strictly to software engineering best practices, resulting in a reliable, secure, and highly functional platform ready to support the next generation of professionals.

References

Github : https://github.com/PavithiraParamakurubaran/Group-2-MAL2020-COMPUTING-GROUP-PROJECT

Youtube Video : https://www.youtube.com/watch?v=1O8zZrG8qn8

Marway, D. ed., (2023). *How to plan a successful quality assurance (QA) strategy*. [online] Medium. Available at: https://medium.com/@dileep_m/how-to-plan-a-successful-quality-assurance-qa-strategy-b423a036bafd [Accessed 25 Feb. 2026].

Gakii, M. and Kihara, A. (2026). Influence of Data Analytics Adoption on The Effectiveness of Strategic Decision-Making in Small and Medium Enterprises in Nairobi County, Kenya. *Journal of Entrepreneurship and Project Management*, 11(1), pp.19–32. doi:https://doi.org/10.47941/jepm.3531.

Sommerville, I. (2011). *SOFTWARE ENGINEERING Ninth Edition*. [online] Available at: https://engineering.futureuniversity.com/BOOKS%20FOR%20IT/Software-Engineering-9th-Edition-by-Ian-Sommerville.pdf [Accessed 21 Mar. 2026].

Wolford, B. (2025). *What is GDPR, the eu's new data protection law?* [online] GDPR.EU. Available at: https://gdpr.eu/what-is-gdpr/ [Accessed 25 Feb. 2026].

Appendices

11.1 Student Portal

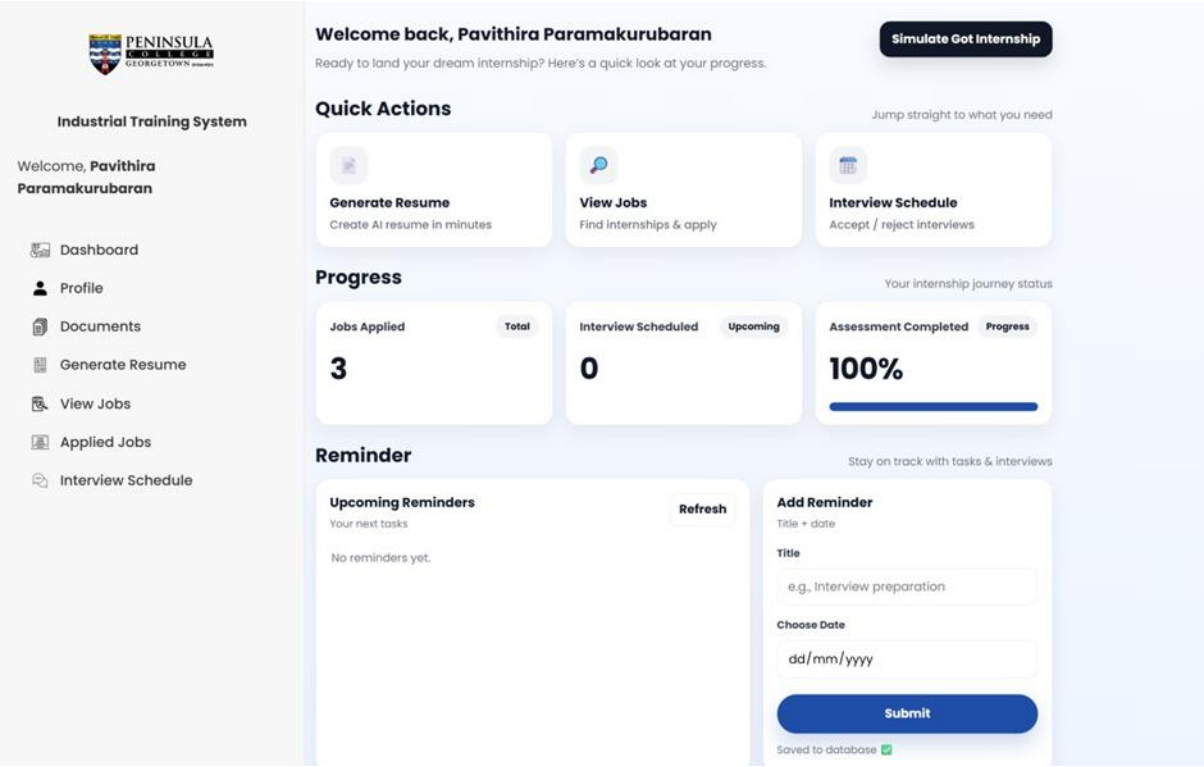


Figure 11.1.1 shows student dashboard

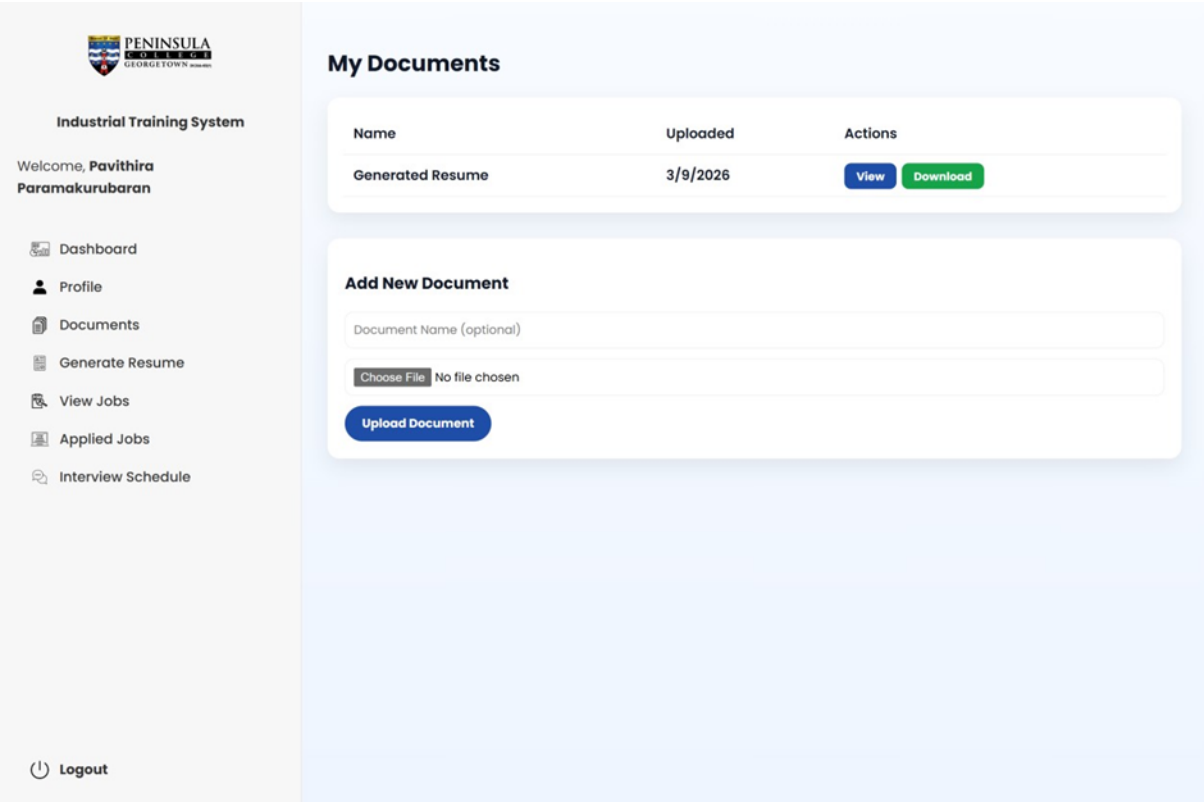
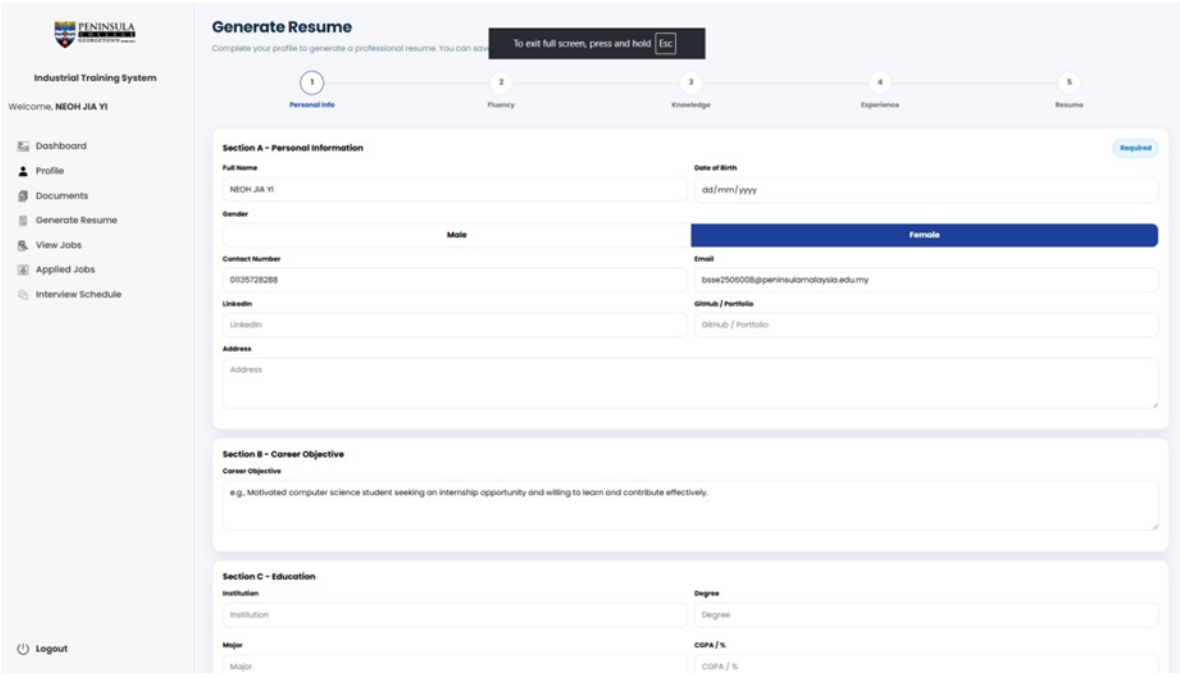


Figure 11.1.2 shows where students upload their important documents



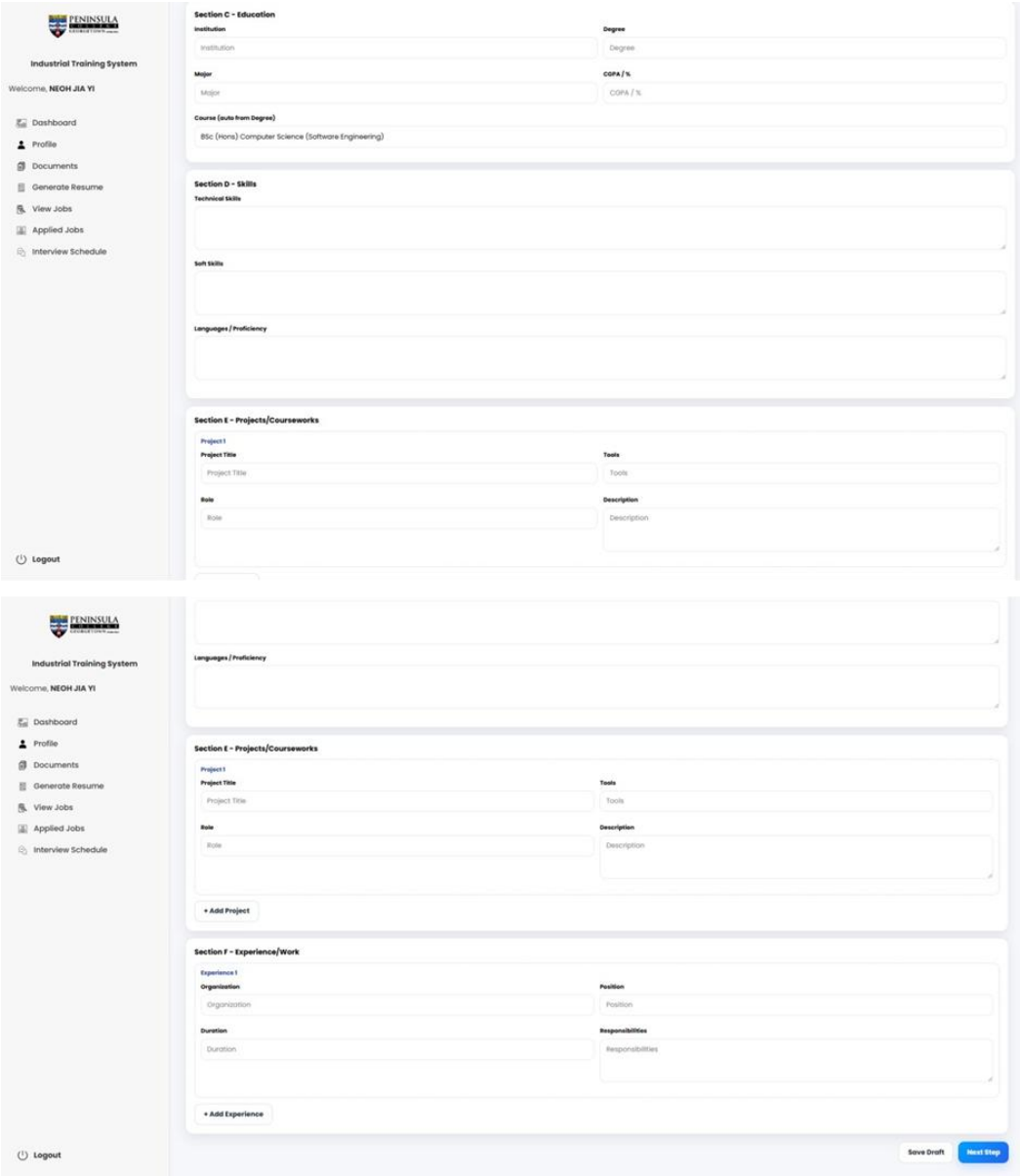


Figure 11.1.3 shows the first step of students generating their resume

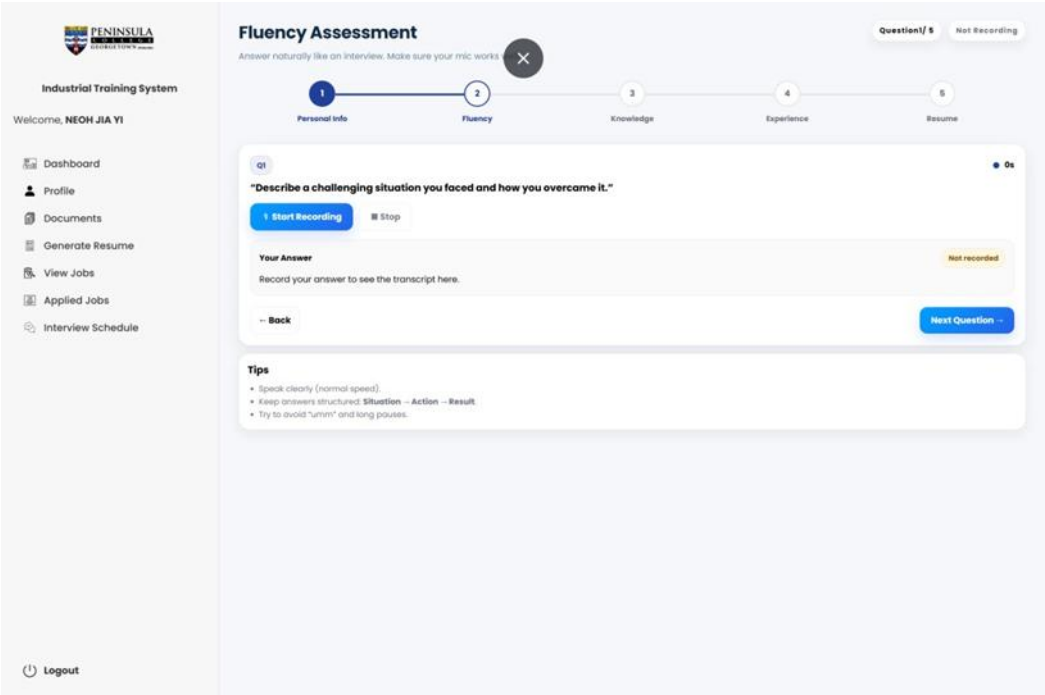
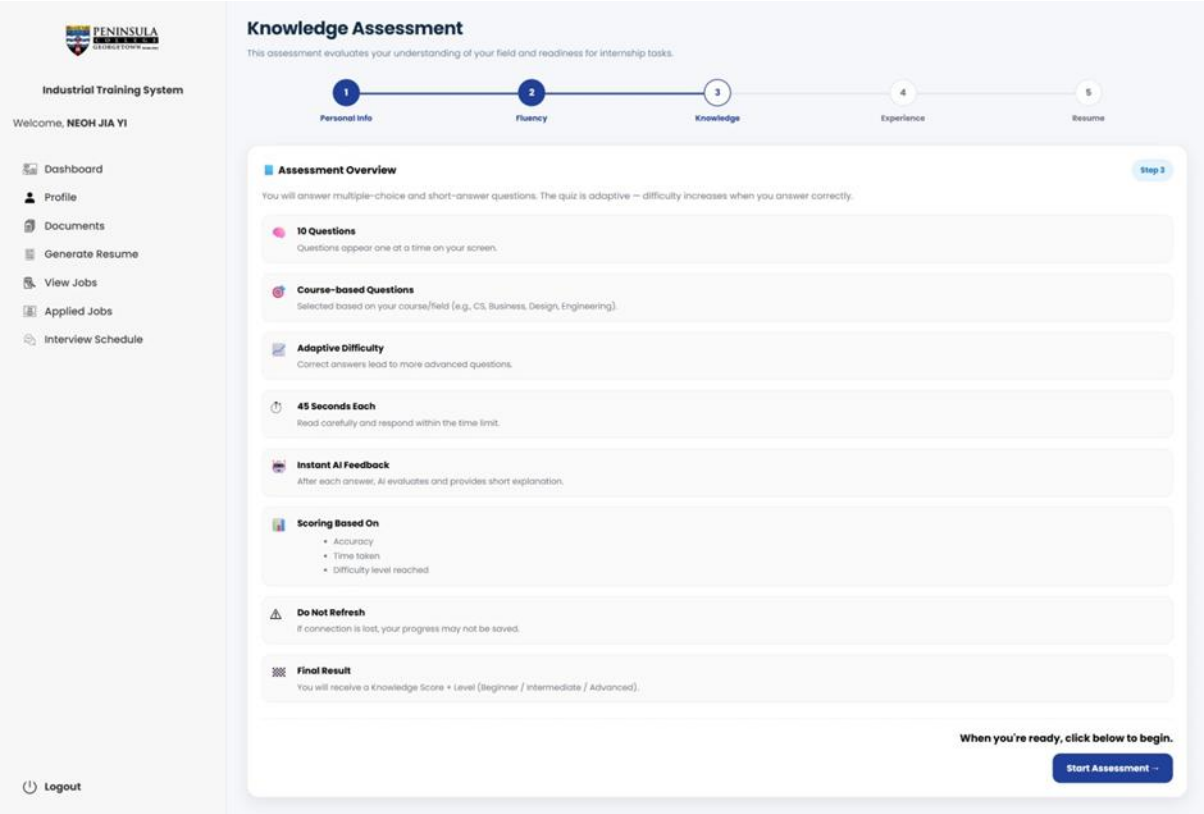


Figure 11.1.4 shows the second step of students generating their resume



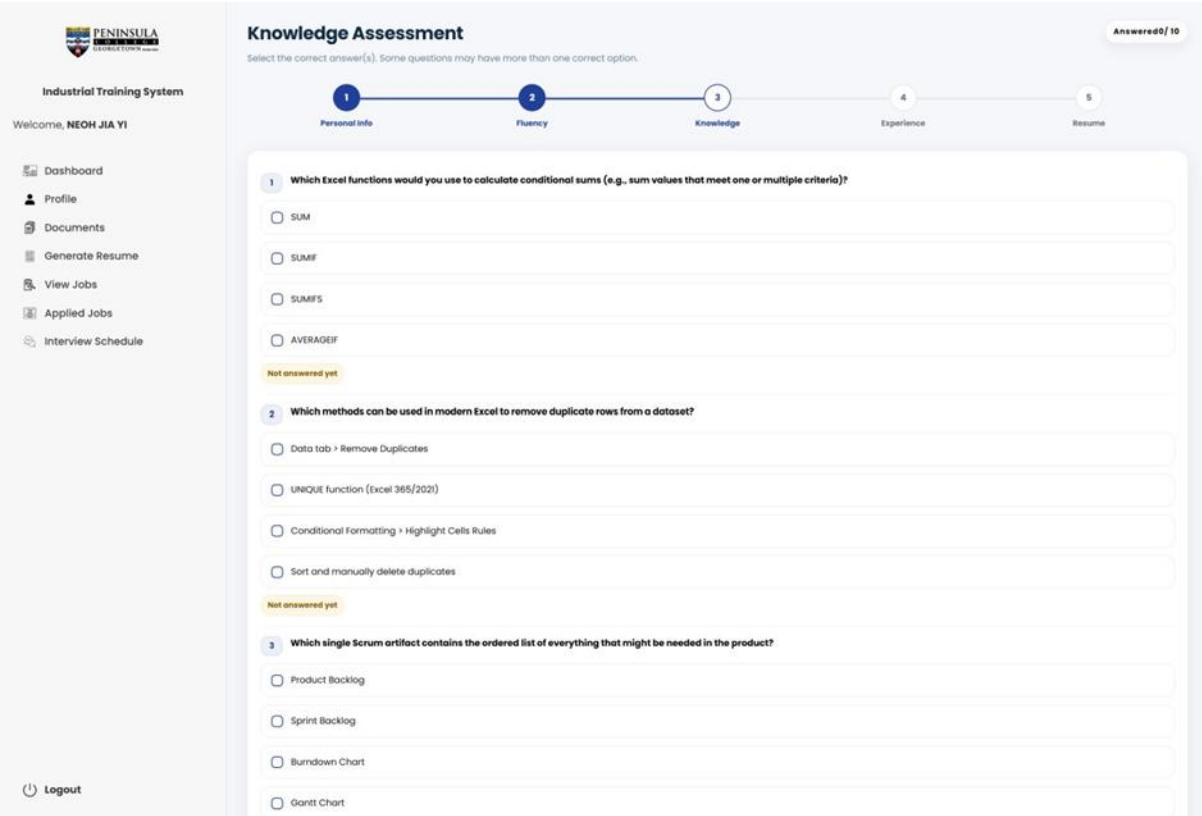


Figure 11.1.5 shows the third step of students generating their resume

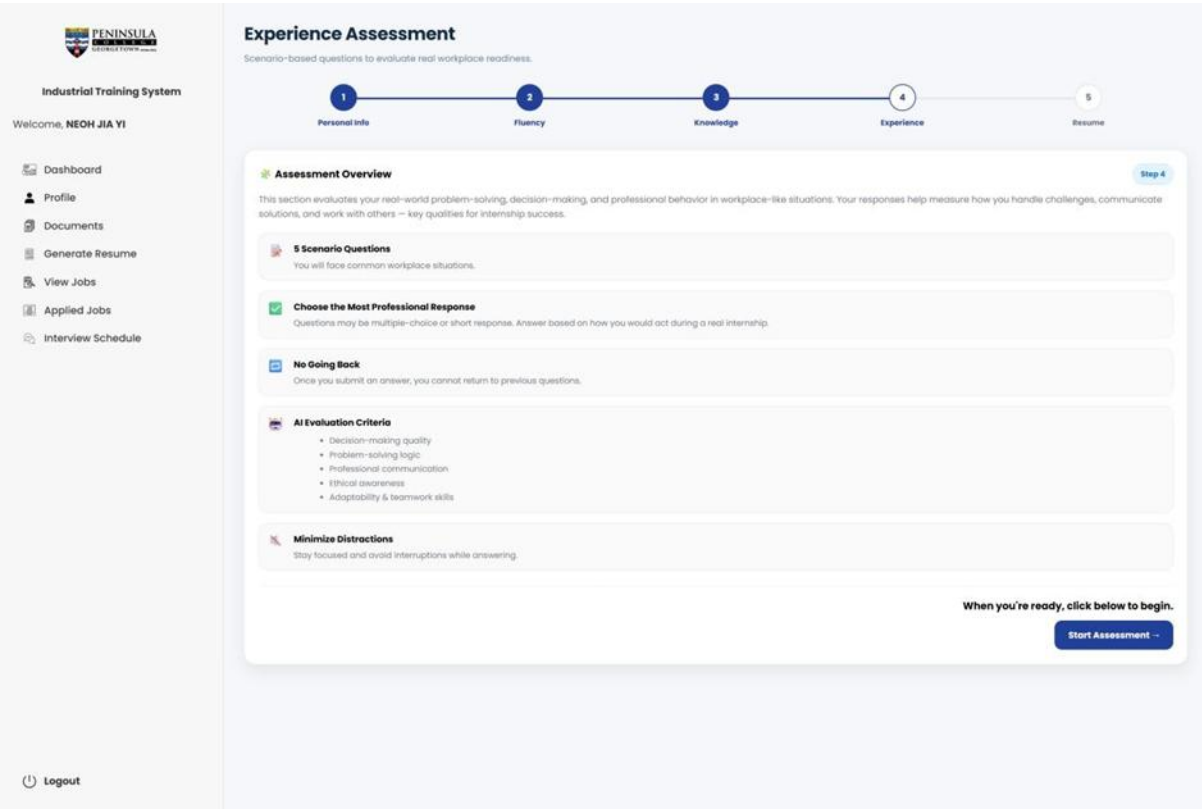


Figure 11.1.6 shows the fourth step of students generating their resume

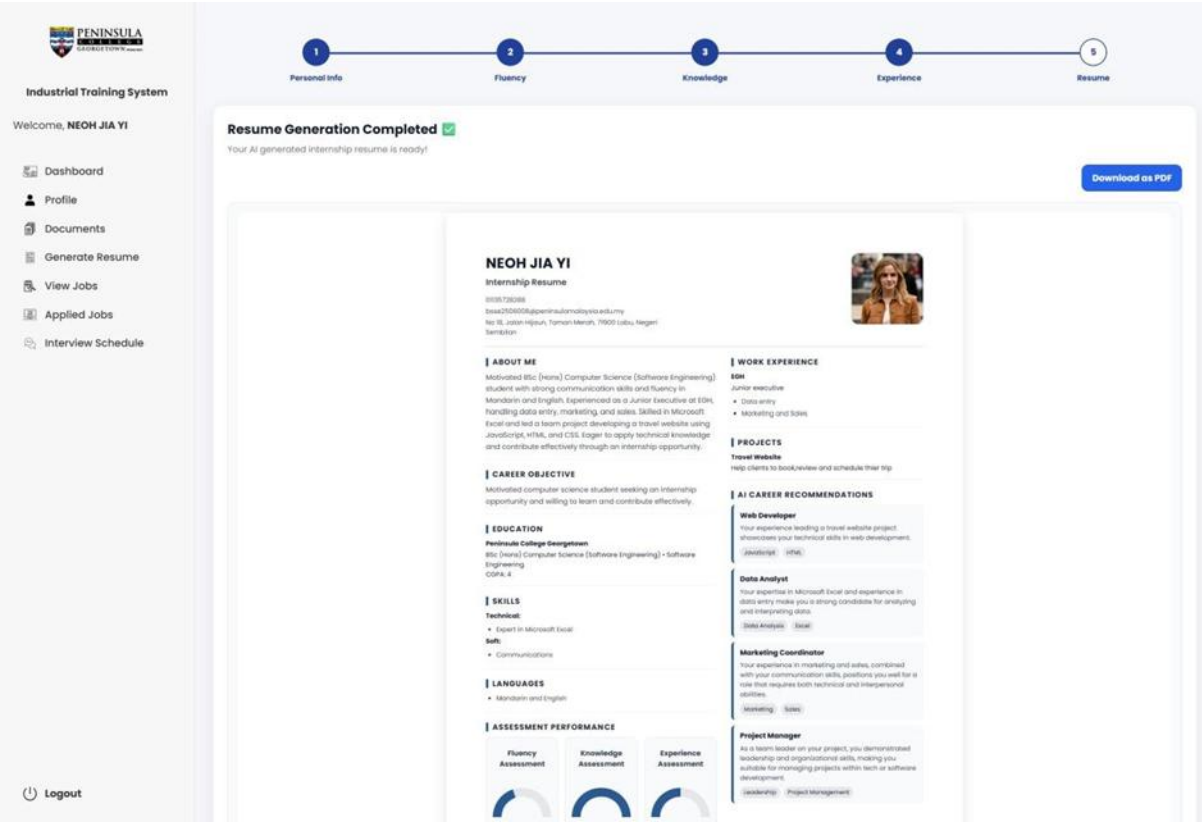


Figure 11.1.3 shows the final step of students generating their resume

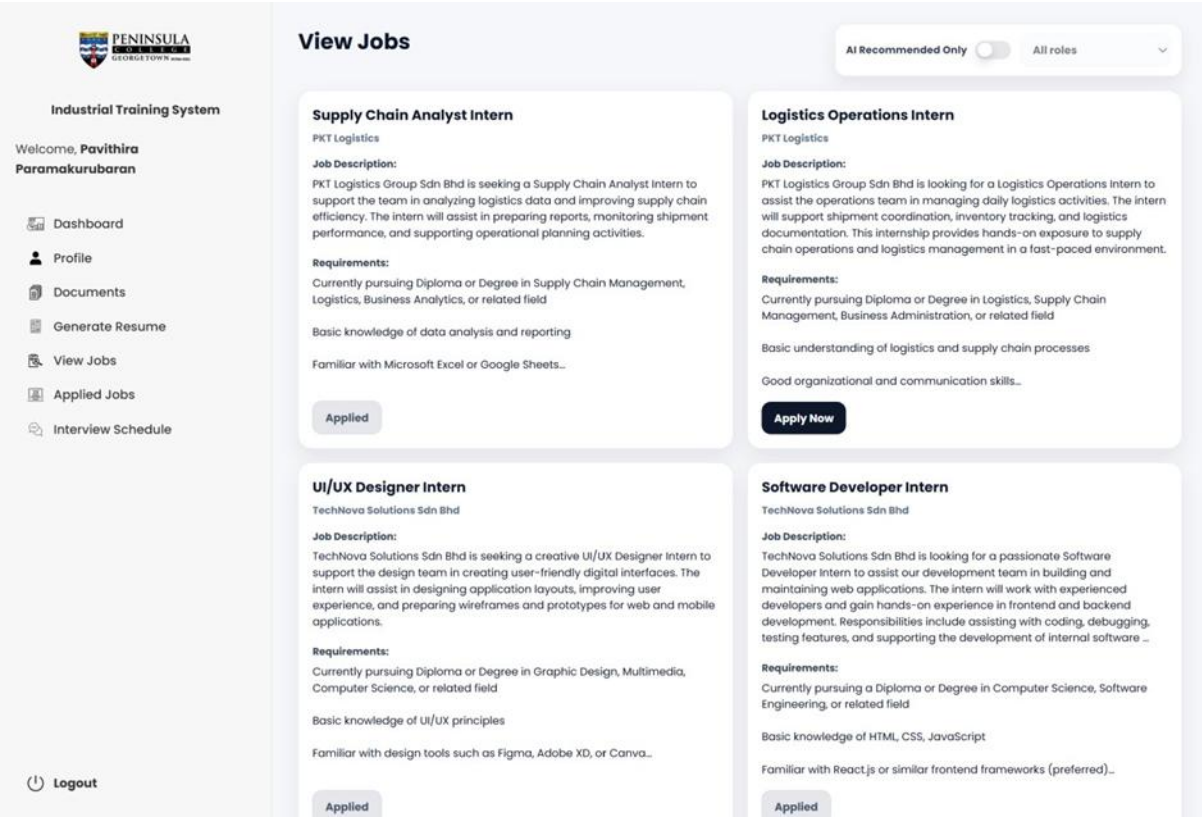


Figure 11.1.4 shows where students view and apply jobs

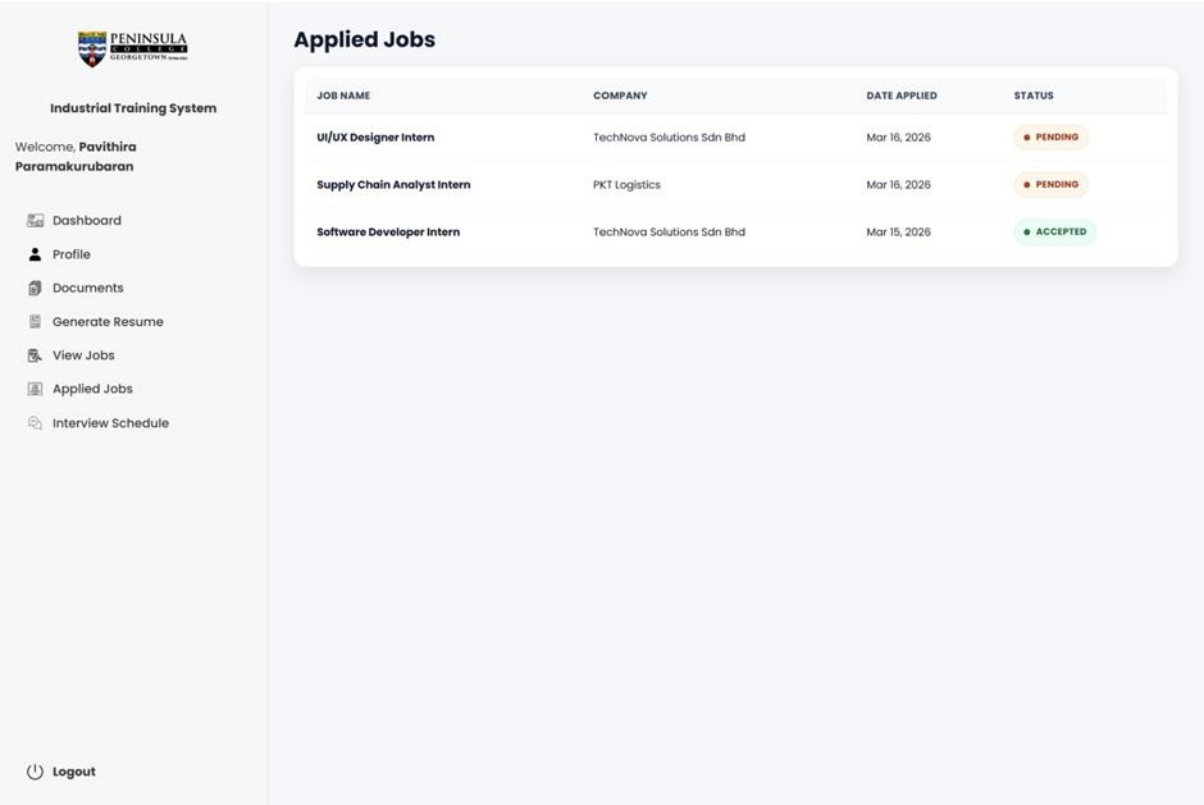


Figure 11.1.5 shows where students view the job they applied

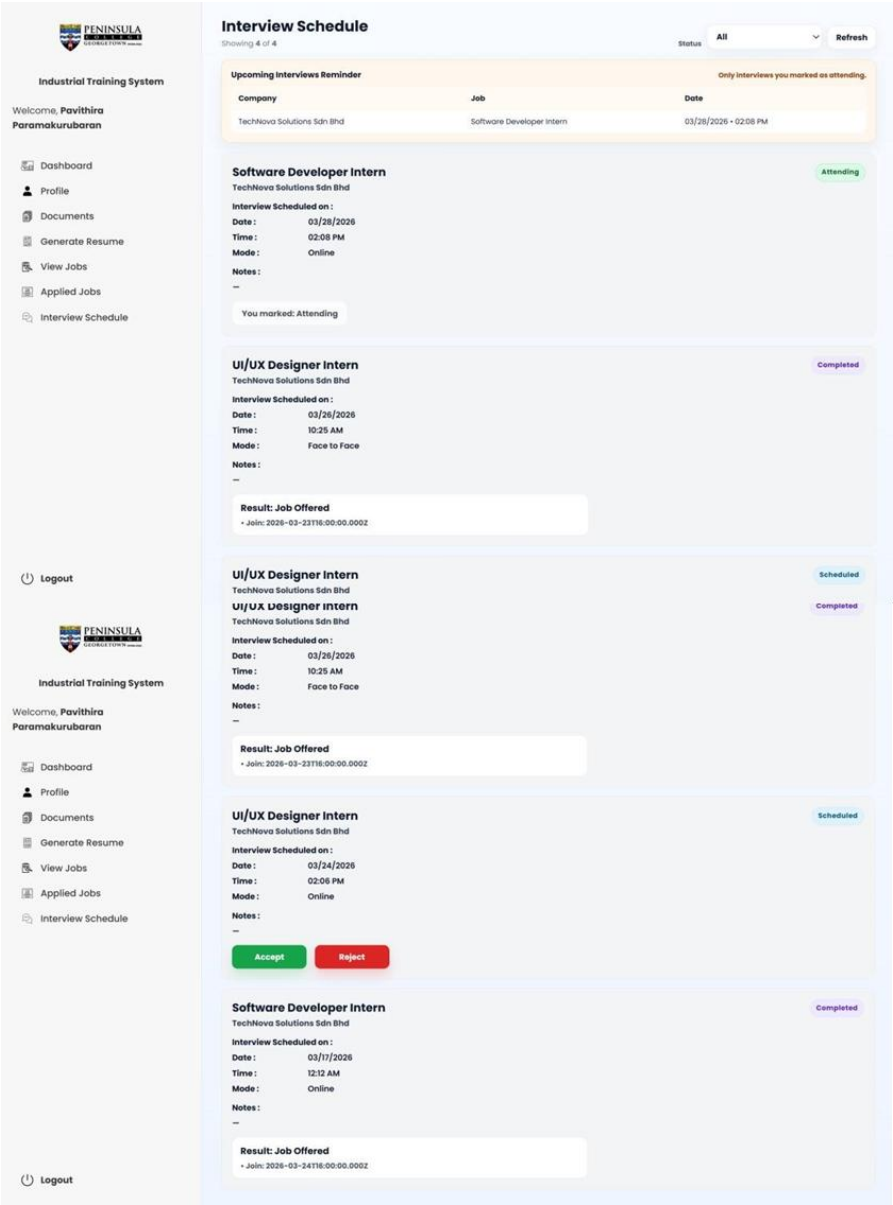


Figure 11.1.6 shows where students view their interview schedule

11.2 Employer Portal

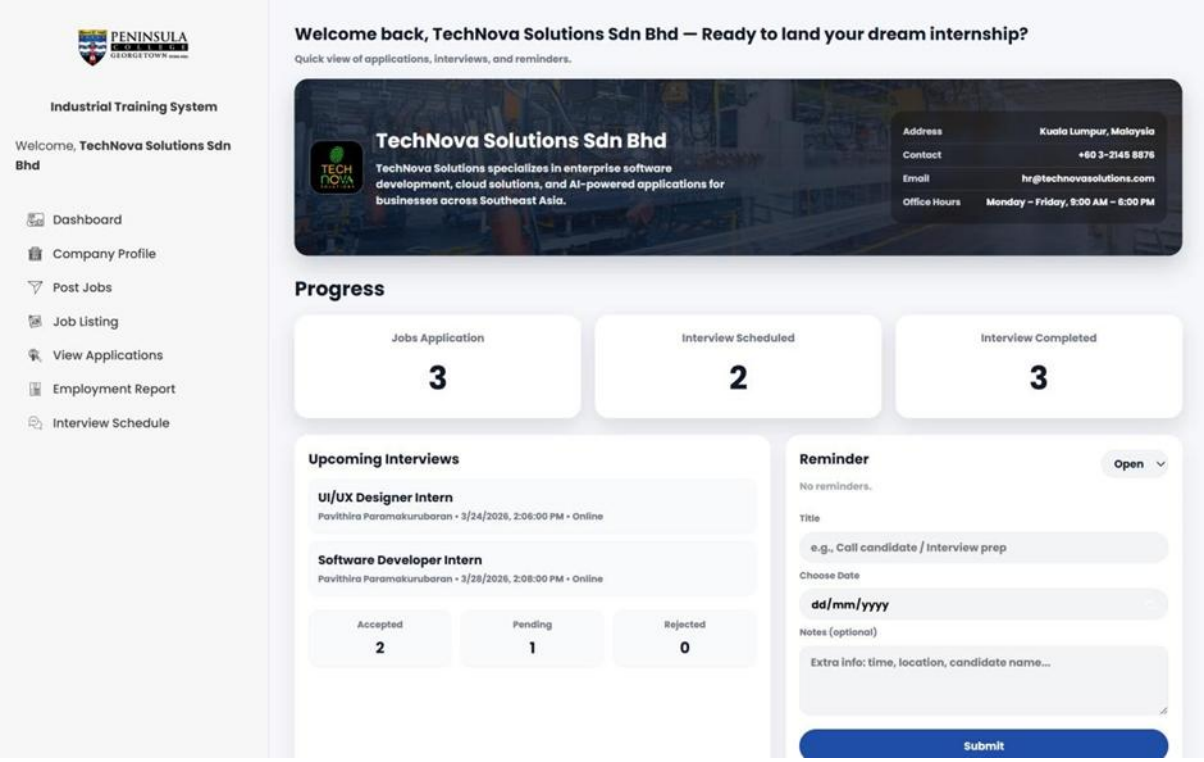


Figure 11.2.1 shoes employer portal

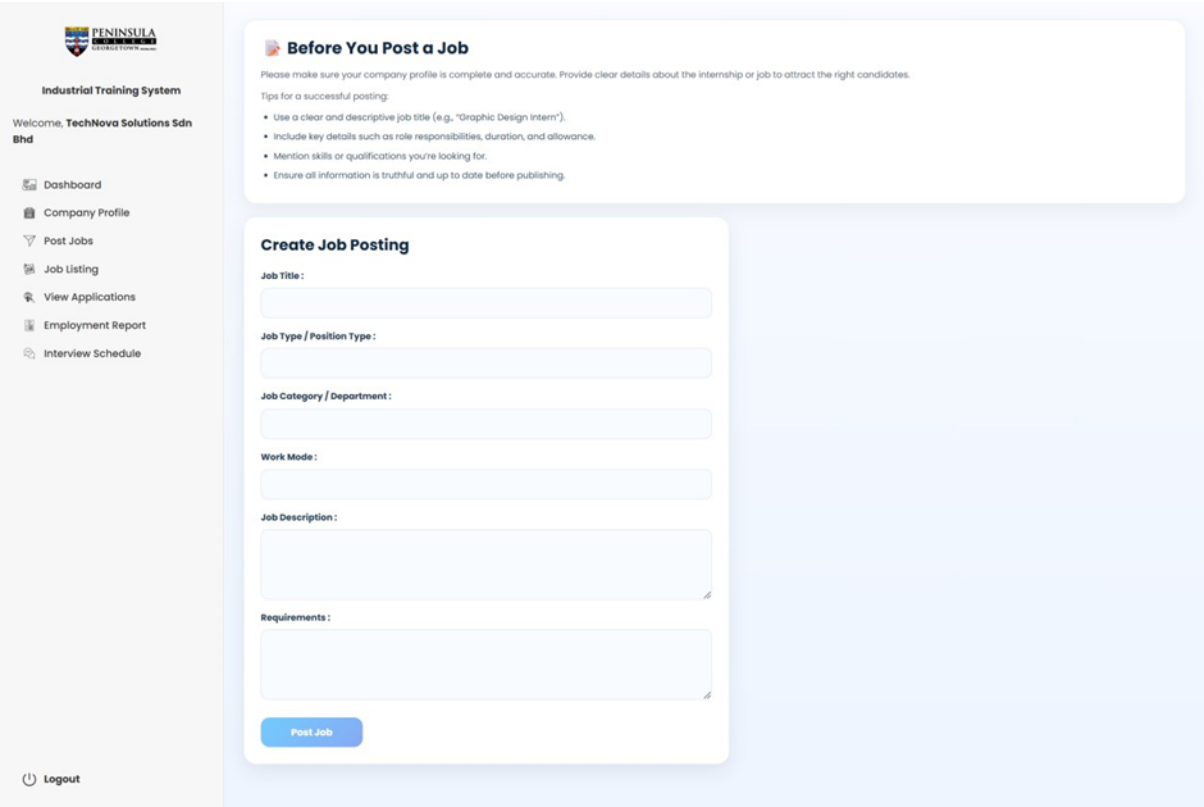


Figure 11.2.2 shows where employer post jobs

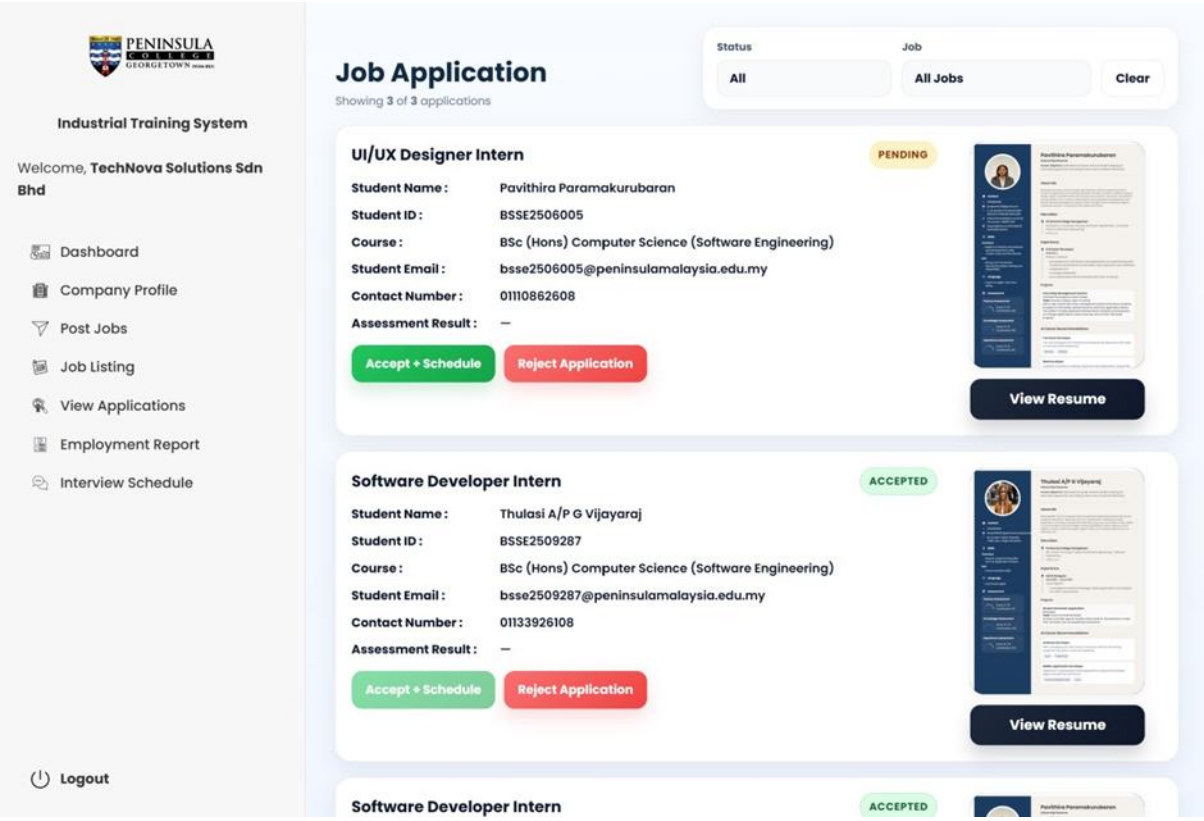


Figure 11.2.3 shows where employer view applications

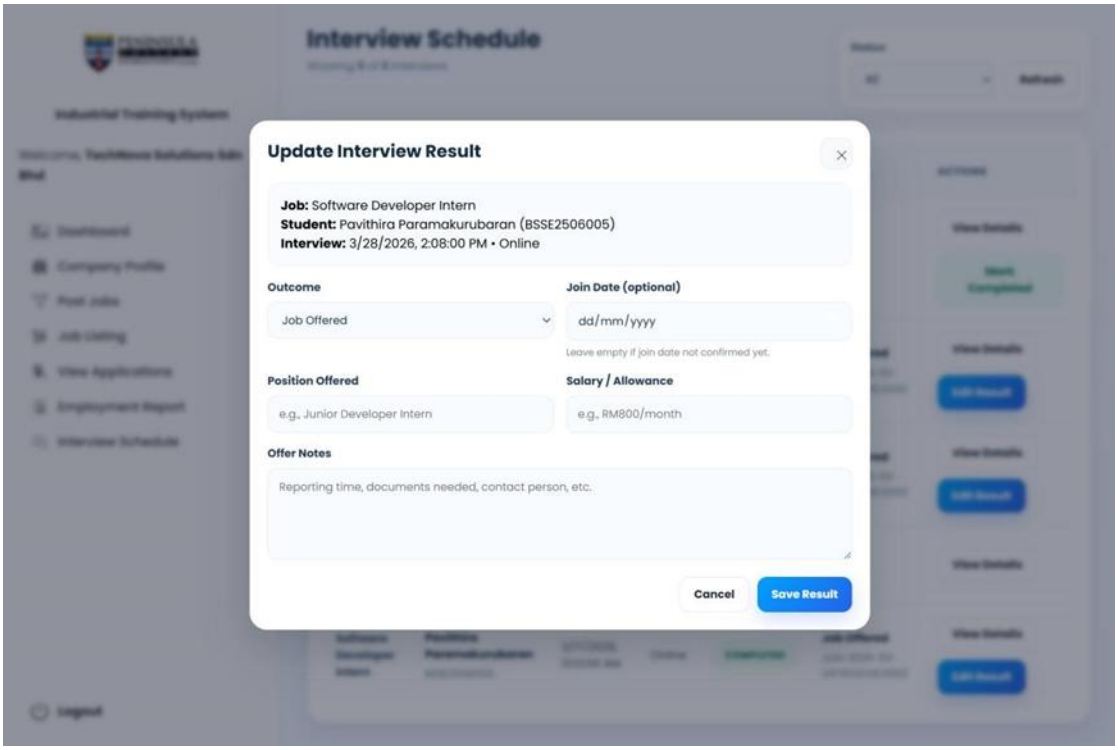


Figure 11.2.4 shows how employer schedule interview

11.3 Admin Portal

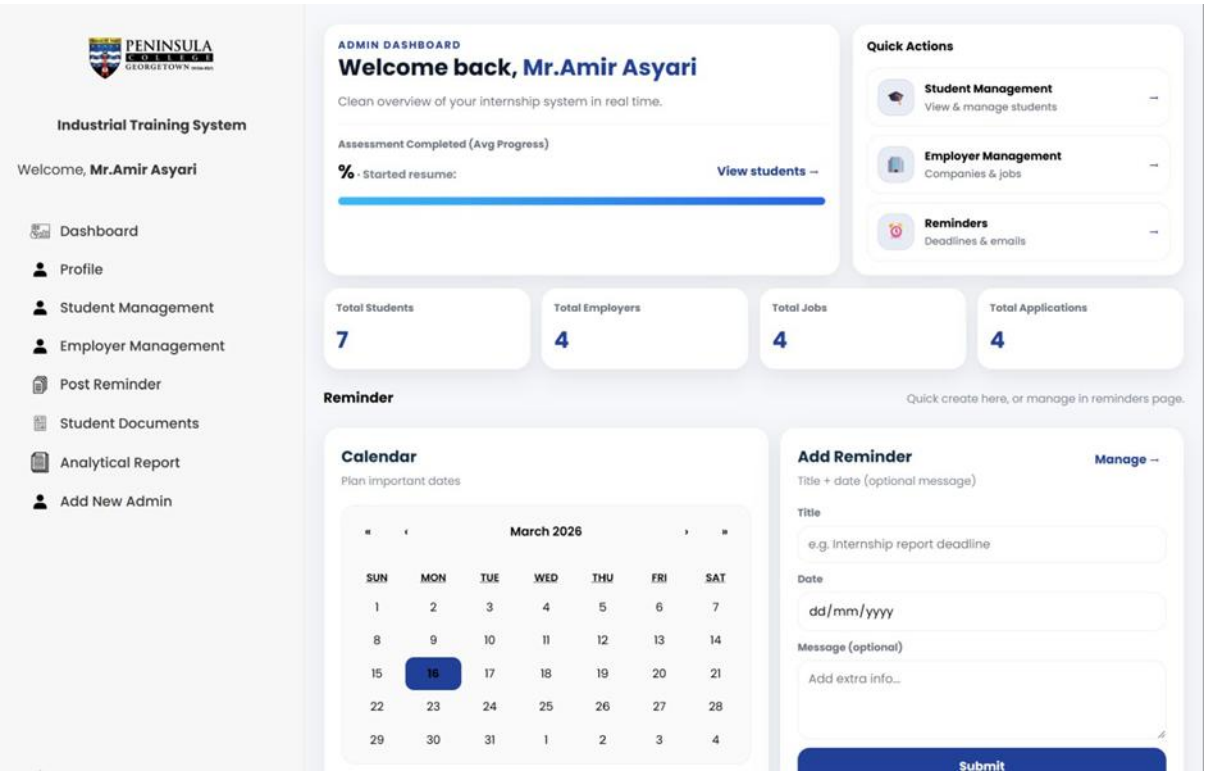


Figure 11.3.1 shows admin dashboard

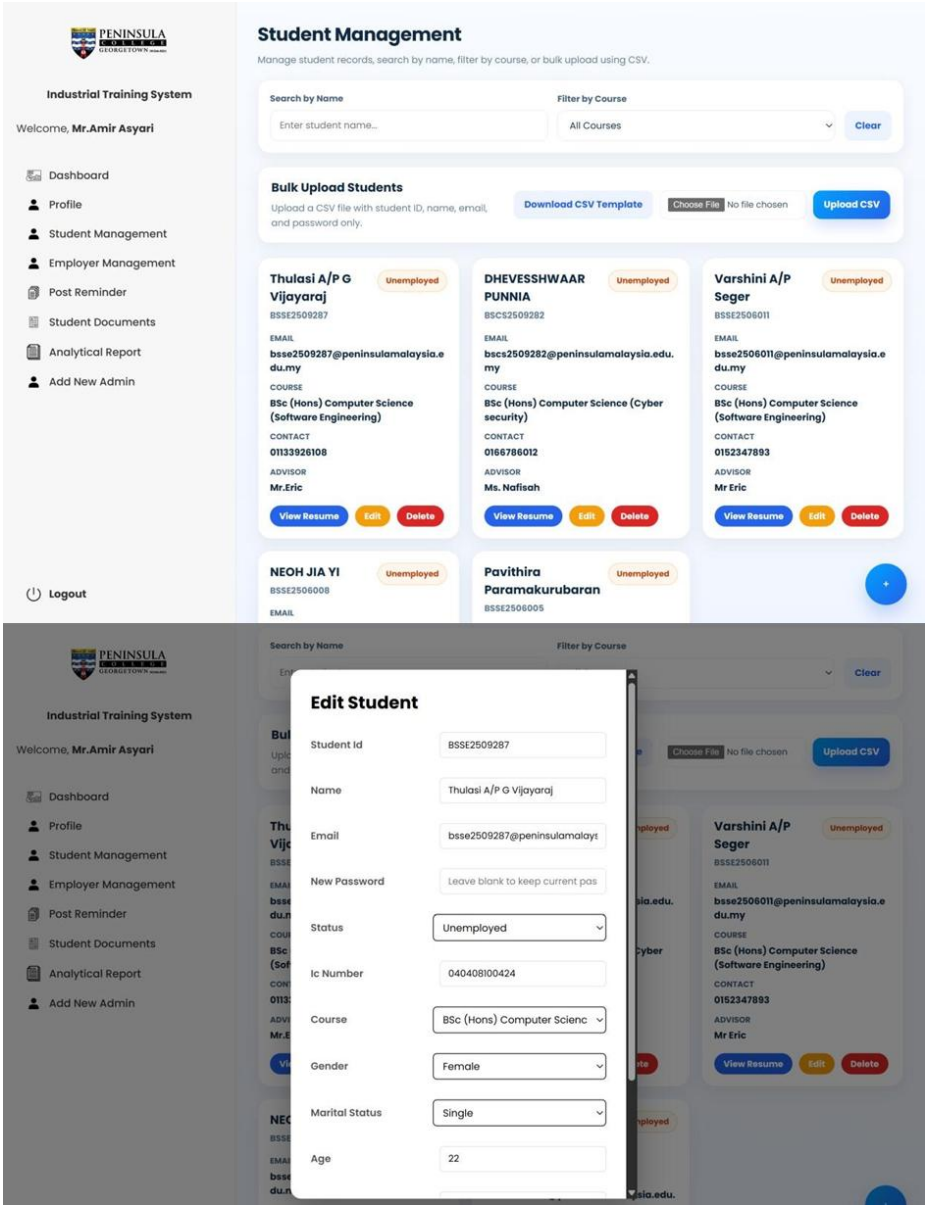


Figure 11.3.2 shows the student management site for the admin

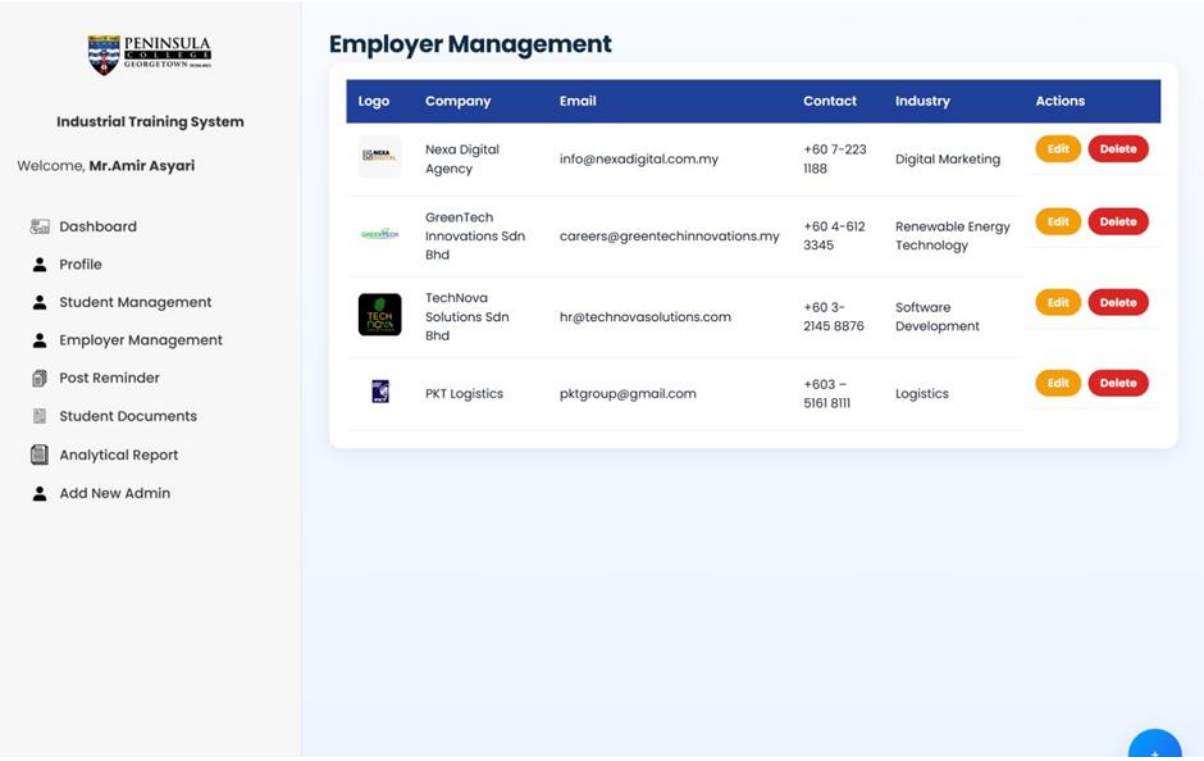


Figure 11.3.3 shows where the employer management site for admin

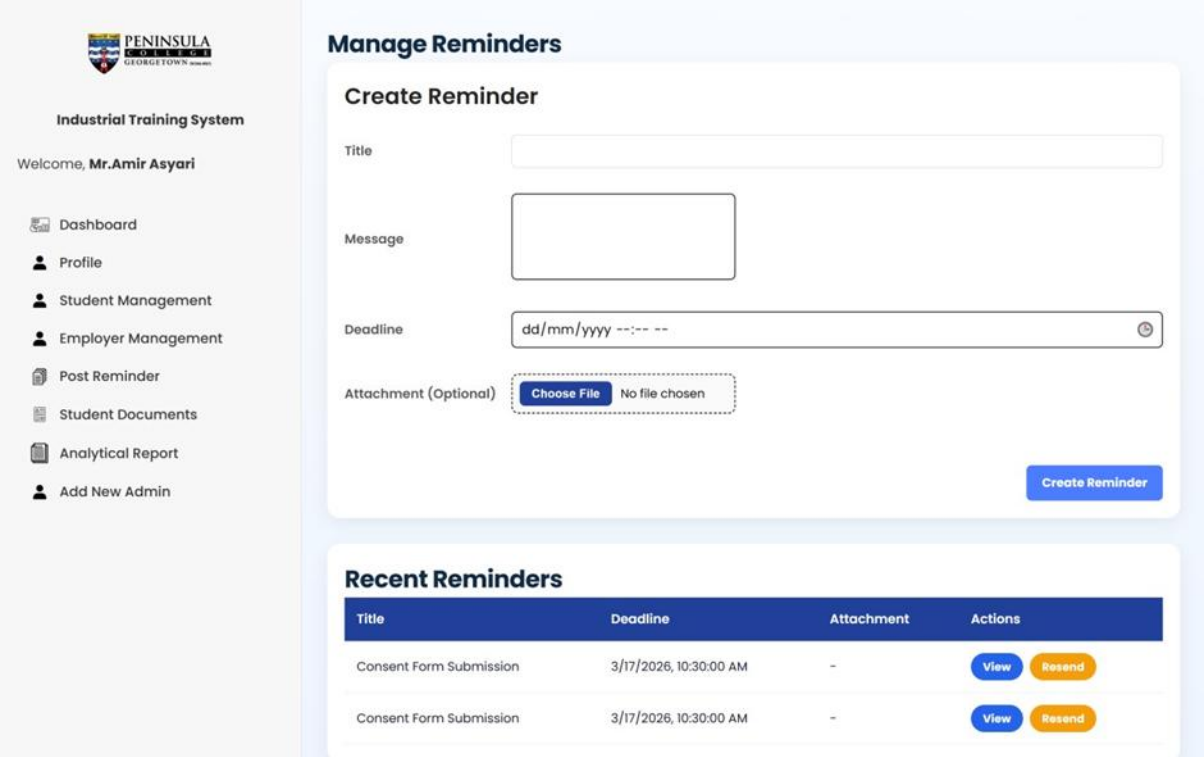


Figure 11.3.4 shows the page where the admin can send reminder to the students

Other Sources

Mockup Link: https://www.figma.com/design/BPPFAIDQMZBOwXaSMOGHLr/Intern-Management- System?node-id=0-1&t=dKa96K3jOAJNqqxd-1

UML Diagram Link :https://shorturl.at/eMnRH

Gantt Chart Link: https://docs.google.com/spreadsheets/d/1S3uhBZuNQVhaLwyEk7H2JapyyFUc2sH2BDCh21OoHNk/edit?gid=0#gid=0

Employer Flowchart Link :https://acrobat.adobe.com/id/urn:aaid:sc:AP:0ce90da4-eb38-4f8a- 9763-5c318930c23f

Admin Flowchart Link: https://acrobat.adobe.com/id/urn:aaid:sc:ap:c5639854-d529-4ff6-979a-8a5a4754b2bc

Student Flowchart Link: https://www.figma.com/design/KaVn58SBDgicw3B29IRIKz/Untitled?nodeid=0-1&t=1XyzhMNeMrRw4yR9-1